

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Music Managment Application

Bachelor's Thesis

KIRILL VOROZHTSOV

Brno, Spring 2026

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Music Managment Application

Bachelor's Thesis

KIRILL VOROZHTSOV

Advisor: Mgr. Luděk Bártek, Ph.D

Brno, Spring 2026



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, the following AI tools were used:

- **ChatGPT:** Used for writing most of the tests, debugging and refactoring, as well as LaTeX formatting.
- **Grammarly:** Used for text corrections.

Kirill Vorozhtsov

Advisor: Mgr. Luděk Bártek, Ph.D

Acknowledgements

I would like to thank Mgr. Luděk Bártek, Ph.D., for his time and the valuable advice he provided for this thesis.

Abstract

This bachelor's thesis aims to develop a web-based music management and streaming platform focused on social interactions between users.

The theoretical part presents a survey that analyzes people's music listening habits, showing that such a platform could attract a solid user base. It then compares existing music-related services with built-in social features and derives functional and non-functional requirements for the proposed system. Then, based on these requirements, suitable technologies and architectural patterns are selected.

The practical part covers the implementation details of the platform, including data model, audio processing, API design, and representation layer.

The result is a fully functional and deployed application.

Keywords

Audio Streaming, Python, JavaScript, MPEG-DASH, HLS, WebSockets

Contents

1	Introduction	1
1.1	Conventions	1
2	Music Consumption Survey	3
2.1	Methods	3
2.2	Results	3
2.2.1	Engagement	4
2.2.2	Listening Methods	4
2.2.3	Social Interactions	4
2.3	Summary	6
3	Existing Platforms	7
3.1	Streaming Services	7
3.1.1	Spotify	8
3.1.2	VK Music	8
3.1.3	SoundCloud	8
3.1.4	Bandcamp	9
3.2	Forums, Blogs, and Other Music Media	9
3.2.1	Rate Your Music	9
3.2.2	last.fm	9
3.3	General-Purpose Platforms	10
4	Specification	13
4.1	Important Terms	13
4.2	Use Cases	15
4.3	Functional Requirements	15
4.4	Non-functional Requirements	17
5	Implementation Planning	19
5.1	Audio Processing and Playback	19
5.2	Authentication	21
5.3	Server-Client Communication	23
5.4	Backend	26
5.4.1	Language	26
5.4.2	Python Framework Comparison	26
5.4.3	Database	28

5.5	Frontend	29
5.5.1	Application Type	29
5.5.2	Language	30
5.5.3	Framework	31
5.6	Summary	31
6	Implementation	33
6.1	Stack	33
6.2	Audio Processing and Streaming	34
6.2.1	Streaming Protocols	34
6.2.2	Packaging	34
6.2.3	Processing Tools	34
6.2.4	Transcoding	35
6.2.5	Audio Preprocessing	35
6.2.6	Backend Pipeline	37
6.2.7	Frontend Playback	38
6.3	WebSocket Communication	38
6.3.1	Backend Handling	38
6.3.2	Frontend Handling	40
6.4	User Management	41
6.4.1	User Data Model	41
6.4.2	Backend	41
6.4.3	Frontend	42
6.5	Account Creation and Authentication	42
6.5.1	Backend	43
6.5.2	Frontend	43
6.6	Songs and Collections	44
6.6.1	Data Model	44
6.6.2	Backend	45
6.6.3	Frontend	45
6.7	Audio Upload/Album Creation	46
6.8	Song Queue	47
6.8.1	Data structure	47
6.8.2	Queue and PlaybackContext	48
6.8.3	QueueSource and lazy filling	48
6.9	Playback and Device Handling	49
6.9.1	Shared state and version counter	49
6.9.2	Position estimation on passive tabs	50

6.9.3	Device activation	51
6.9.4	Browser autoplay handling	52
6.9.5	Frontend playback control	52
6.10	Publications	53
6.11	Search	54
6.12	Notifications	54
6.13	Security	55
6.14	Deployment	55
A	Survey Questionnaire	57
B	Audio Processing Pipeline Details	65
B.1	Streaming Protocols	65
B.2	Packaging	65
B.3	Processing Tools	66
B.4	Transcoding	66
B.4.1	Why transcoding is needed	66
B.4.2	AAC encoder choice	67
B.4.3	Bitrate strategy: CBR versus VBR	67
B.4.4	Generation loss	68
B.5	Audio Preprocessing	68
B.5.1	Input validation limits	68
B.5.2	FFprobe invocation	69
B.5.3	Tier selection	70
B.5.4	Channel normalization	70
B.5.5	Sample rate normalization	70
B.5.6	Bit depth conversion	71
B.5.7	Loudness theory	71
B.5.8	Why loudness is measured after Transcoding	72
B.6	Pipeline invocations	72
B.6.1	FFmpeg transcoding flags	72
B.6.2	Shaka Packager flags	73
B.6.3	ebur128 loudness measurement flags	75
C	Diagram Catalog	76
C.1	Use Case Diagrams	76
C.2	Frontend Component Hierarchies	82
C.3	Behavior Diagrams	84

C.4 Survey Charts	85
Bibliography	90

List of Tables

3.1	Comparison of Service-Specific Social Features	11
-----	--	----

List of Figures

3.1	Music Discovery Methods Popularity	7
C.1	Use cases: Account and Identity	76
C.2	Use cases: Social	77
C.3	Use cases: Library and Content	78
C.4	Use cases: Playback	79
C.5	Use cases: Search	80
C.6	Use cases: Communication	81
C.7	Frontend component hierarchy: top level	83
C.8	WebSocket cache invalidation on follow mutation	84
C.9	Age Distribution of Respondents	85
C.10	Monthly Song Count per Respondent	86
C.11	Listening Frequency of Respondents	86
C.12	Audio Media Consumption by Age Group	87
C.13	Streaming Platforms Popularity	87
C.14	Social Interaction Methods	88
C.15	Social Interactions Frequency	88
C.16	Music Sharing Frequency	89
C.17	Music Sharing Methods	89

1 Introduction

Music streaming has been steadily growing in popularity each year for the past decade. [1] And this is to be expected; numerous studies have demonstrated how important music is for people. For instance, Rentfrow [2] suggests that music can influence mood, change self-perception, and help with social bonding. In addition, the rise of the clubbing scene, music festivals, and other social music-related activities clearly indicates the important social role of music. [3]

However, despite this social significance, it is surprising that features which facilitate social interactions are still not widely implemented in the existing music streaming platforms, as will be shown in **Chapter 3**.

The goal of this thesis is to design a music-centric platform that provides capabilities for collaboration and social interaction around music.

This work is divided into the following **six** chapters:

- **Chapter 2** Presents the outcomes of a survey illustrating how people consume music, how prevalent it is in social interactions, and why this type of platform can be relevant.
- **Chapter 3** Explores which social features existing streaming platforms and other music-related services already offer.
- **Chapter 4** Outlines the functional and non-functional criteria for the application.
- **Chapter 5** Describes the choices of technologies that are used by the application.
- **Chapter 6** Explains the development process and implementation details.
- **Chapter ??** Summarizes the results of the work.

1.1 Conventions

Some diagrams in this thesis are too large to embed inline. They are referenced by their file path in the project repository, for example

`text/diagrams/HighLevelSystem.png`. The complete set is kept under `text/diagrams/`.

File and directory paths are written with Identifiers, such as class names, function names, and package names, use

2 Music Consumption Survey

In order to better support the choice of topic and demonstrate that a music streaming platform centered on social interactions could be a relevant service, a brief survey was conducted. It examined individual music listening and discovery habits, platform usage patterns, music-related interactions between people, and attitudes towards potential social features on the platform.

2.1 Methods

The service chosen for the questionnaire was Google Forms [4]; it provides a simple interface for survey creation, allows for easy sharing of the form, and supports extracting results as CSV files.

The questionnaire consists of 15 questions. Most of them are multiple-choice and closed-ended, with some allowing a custom answer. All custom answers are grouped under the “other” category in the provided figures. Answers in their original form can be found in the full survey.

The complete questionnaire is included in the thesis repository under `text/extras/survey-questions.pdf` and online.¹

2.2 Results

This section presents the survey outcomes and analyzes their significance. Not all questions are discussed; for example, some questions dedicated to music discovery are omitted.

A total of 119 people participated, with the majority being in the 18 to 30 age group, with the exact distribution shown in Figure C.9.

1. https://docs.google.com/forms/d/1vhhAu_SfuHV4xy6JoDaRsM5uCElYn_csTmN8u4ZlpHc

2.2.1 Engagement

Firstly, it was necessary to determine whether respondents listen to music frequently and whether they use streaming platforms for this purpose.

Over 50% of respondents reported listening to more than 500 songs per month, i.e., 16 songs a day on average (Figure C.10), and nearly 80% stated that they listen to music daily (Figure C.11).

The numbers clearly indicate that people have a significant interest in music, and the next logical step was to determine how the audio content is predominantly accessed.

2.2.2 Listening Methods

As shown in Figure C.12, the proportion of respondents using streaming platforms across all age groups is approaching 100%, with slightly higher numbers in the younger age groups. Although downloaded files and physical media are still used, especially among people aged 30-60, they are usually only a secondary option next to streaming solutions.

The question dedicated to the popularity of individual platforms has shown that Spotify is the most used one, which aligns with global statistics [5]. However, Yandex Music, coming in second place, deserves further explanation. Most of the respondents were from Russian-speaking countries, specifically Russia, and, with many Western companies leaving its market in 2022, including music streaming services, most users have moved to locally available products: Yandex Music, VK Music, and Zvuk. This is also the reason, why other popular region-specific platforms, such as Amazon Music (US), QQ Music (China), and JioSaavn (India), are absent from the survey.

The complete statistics are shown in Figure C.13.

The survey showed that streaming services are indeed widely used. The next important step was to analyze the regularity of music-centered social interactions.

2.2.3 Social Interactions

The results of Question 10 indicate that nearly all respondents (99.2%) reported listening to music with others at least sometimes, primarily

in offline settings such as gatherings or car rides (Figure C.14). Online collaborative listening methods, although less common, were also mentioned by a quarter of respondents.

Question 11 (Figure C.15) examines the frequency of these interactions: the results show that a significant portion of respondents engage in shared listening regularly. Around 60% reported doing so at least a few times per month, with a smaller group indicating almost daily interactions. This supports the previously mentioned idea in **Chapter 1** that music consumption is a common social activity.

The next question is phrased as follows: “How often do you/your friends share/discuss music?”. While the previous question focused on real-time in-person collective listening, this one highlights more intentional and in-depth musical interactions, such as sharing songs, recommending music, or discussing it. Music, in that case, is not just the background but the primary purpose of engagement. The same trend as before can be observed in the responses: over 70% (Figure C.16) indicated that they share music at least several times a month, with many doing so weekly or more often. This further emphasizes the active social role of music.

Question 13, “How does that happen?” (in terms of music sharing), asks about the specific ways people share music. Many respondents (Figure C.17) stated that they typically send links from streaming platforms through external messaging apps. While this shows an apparent demand for sharing music online, it also reveals a gap among platforms, as these interactions remain fragmented. Hence, enabling such exchanges directly within the streaming app could provide a smoother and more uniform experience.

Responses to the question “Would you say that you know a lot of people with similar music taste?” were mixed, with a near-even split. This suggests that while some users already have a social circle with similar music taste, many do not. In addition, when asked “Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?”, the majority answered positively.

This data further support the idea that social discovery features could be useful within a streaming platform. Moreover, over 60% of participants stated in Question 15 that they would use additional social features if available on a streaming platform.

2.3 Summary

Overall, the results indicate that people listen to music frequently, with streaming services being their primary medium of accessing it. The data also suggest that music plays an important role in everyday life and often appears in social situations.

Yet, existing platforms only partly support interaction between users. This makes the idea of a music streaming service with built-in social features both relevant and likely to address real user needs.

3 Existing Platforms

In this chapter, existing platforms and services are examined to gain a better understanding of the instruments people use when interacting with music. Additionally, this chapter provides insight into the potential features that could be implemented in the resulting application.

Firstly, streaming services are discussed, as the main aim of the thesis is to create a system for music streaming. Then, other music-related services (e.g., forums) are presented, as they offer alternative possibilities for engaging with audio content that may be beneficial for the future platform. Lastly, platforms such as YouTube and TikTok are examined, because the survey has shown that they are a popular way to interact with music 3.1.

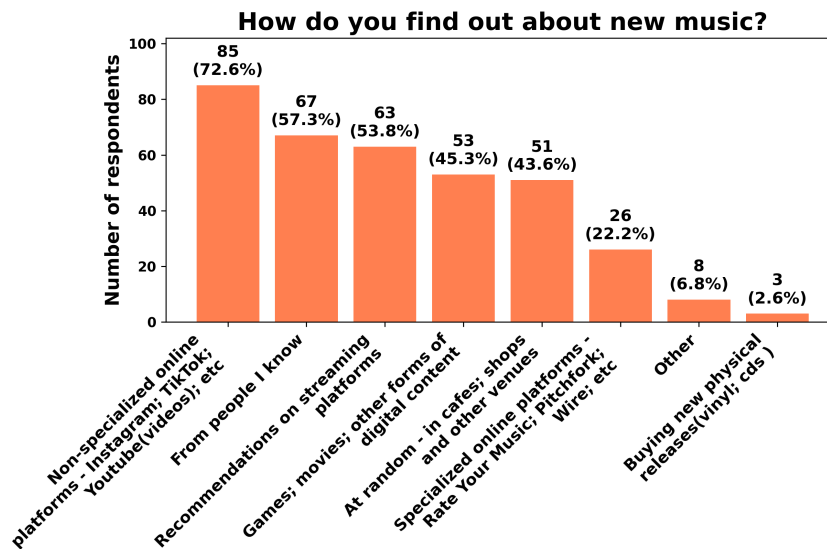


Figure 3.1: Music Discovery Methods Popularity

3.1 Streaming Services

As already illustrated in Figure C.12, and further confirmed by the study of the International Federation of the Phonographic Industry [6],

the most common way of music consumption and discovery is through streaming services. Although many options are available, this section only considers those that are both popular and have unique features. The descriptions, rather than offering a general portrayal of each platform, will focus on the service-specific social features.

3.1.1 Spotify

One of the most prominent social features on Spotify is the “Spotify Jam” feature [7]. It lets people create a collective song queue that is then synchronized among all connected users. Moreover, volume, the order of songs, and other playback aspects can be controlled individually by each user.

Another notable feature is “Friend Activity” [8], the followers of a given user are currently listening to.

The “Listening Parties” feature also deserves a mention: these are live chats with a limited capacity that can be joined for a short time when new music is being released [9, 10]. This somewhat brings the offline collective music discovery experience to the online world.

During the implementation of this thesis (the text was initially written in spring–summer 2025), Spotify has also added support for user-to-user chats (from September 2025) [11]. This further supports the claim that social features are in demand on music platforms.

3.1.2 VK Music

VK Music is a music service integrated into the VK social network. Consequently, it is possible to send songs and playlists via private messages or add them to posts in groups. VK Music also provides an “Updates” tab, which is populated by the audio materials that any followed user or group has added to their “Liked Feed” [12].

3.1.3 SoundCloud

SoundCloud is one of the few platforms that lets its users leave comments and reactions on songs and playlists [13, 14]. In addition, each user has a feed consisting of personal uploads and reposts of others’ content [15], which is visible when visiting their profile.

3.1.4 Bandcamp

Every Bandcamp user has a profile with four tabs: “collection”, “wishlist”, “followers”, and “following”. The most interesting feature is under the “following” tab: users can subscribe to available genres and then specify the ones they want to showcase to others viewing their profile. The “wishlist” tab is also noteworthy, as in this tab the users can showcase what they are looking forward to listening to in the future.

This concludes the streaming services section. Services that are focused on music, but not on the streaming itself, are discussed next.

3.2 Forums, Blogs, and Other Music Media

Different resources are gathered under these umbrella terms, from professional music review websites to amateur and personal pages. Again, only widely used services that offer something noteworthy are listed.

3.2.1 Rate Your Music

“Rate Your Music is one of the largest music databases online. It is an incredible tool that can help you find and learn about new music to listen to.” [16]

This website is one of the biggest platforms with user-created content. Every member can write reviews and set ratings for music releases, contribute to the “wiki” consisting of genres, thematic lists, and charts, and connect with other members of the forum.

3.2.2 last.fm

Last.fm is primarily a listening statistics and music discovery service. However, it also has a couple of notable social features:

1. A global and personal events pages. In the former, users are able to browse the upcoming events near their location, and in the latter they are able to showcase which events they plan to

attend. Other users can then see, either on the event page or on that user's profile, which events the user is interested in and/or plans to attend.

2. A "Neighbours" tab. The "Neighbours" tab shows other users with similar listening habits, based on shared artists and an overall compatibility score.
3. "Shouts" section. The "Shouts" section is a profile-scoped feed, where the users can leave comments and reply to what others have written.

This section has introduced a couple of possible features not previously seen in the listed streaming services, such as discussion forums and profile feeds.

The following section examines platforms that are not primarily focused on music but still influence how people engage with it.

3.3 General-Purpose Platforms

As mentioned earlier, one of the most interesting results of the survey is the high number of people discovering music on platforms that are not focused on music. Services like Instagram, YouTube, and TikTok, which are primarily used for sharing videos and other user-generated content, have become closely tied to how people find music and listen to it. In recent years, these platforms have started to promote music more directly by allowing users to embed tracks within their platform-native content.

Instagram, for example, added a special audio tool that lets users include music in their short videos ("Reels" [17]), helping songs reach more people through visual content.

YouTube automatically identifies songs used in videos and displays them in the description, making it easier to find and listen to them.

TikTok has had an even bigger impact: many songs have gone viral and later became hits in charts and streaming apps because people use them in popular videos (these are often called "TikTok Hits").

This shows that big platforms are not just reacting to users' interest in music, they actively help promote it by connecting songs with content that people want to watch and share.

An integration with such services can be a big plus for the platform. A possible feature could be a badge stating “This track was in a TikTok user Alice123 sent you today!”.

Summary

The table 3.1 summarizes the comparison between the features mentioned in the sections above.

Difficulty suggests the possible complexity of implementation.

Mode assumes the typical context in which the feature operates.

Social Interaction Level indicates user involvement when using the feature.

Table 3.1: Comparison of Service-Specific Social Features

Feature	Interaction Level	Mode	Difficulty
Chat	Very High	Pair	High
Common Listening	High	Group	Very High
Forums/Wiki	High	Group	Medium
Personal Feed	High	Individual	Low
Song/Playlist Comments	High	Group	Low
Listening Parties	Medium	Group	Very High
Post Embeddings	Medium	Group	Medium
Friends’ Listening Activity	Medium	Pair	Low
Following Genres Display	Low	Individual	Medium
Real-Life Events Section	Low	Group	High
Music Wishlist	Low	Individual	Low
External Services Integration	Low	Pair/Group	High

Overall, it is clear that numerous music-related platforms with social features built in are available across the Internet. However, services that focus on the actual audio delivery often support just a subset of the possible features. This, in turn, forces users to use multiple platforms in order to meet their needs. This project aims to bridge that gap, gathering multiple social features in one app.

Based on the research done in the previous and current chapters, the following social features were chosen for implementation, as they provide a good balance between the complexity of implementation and the level of social interaction:

- Chat.
- Personal/Global Feed.¹
- Song/Playlist Comments.
- Friends' Listening Activity.

These and further requirements for the application are listed in the next chapter **Chapter 4**.

1. In comparison to `last.fm`, a common view with posts from multiple authors is going to be implemented as well.

4 Specification

In this chapter the general outline of the application is given via use cases and functional/non-functional requirements. The individual requirements are constructed based on the 'MoSCoW' criteria [18]. "Must Have" requirements specify functionality without which the application can not be considered finished. Those are the main features and technical aspects that must be implemented. "Should Have" requirements mostly represent features that would greatly improve the user experience, but are not strictly needed for the application to be fully working. "Could Have" requirements are reserved for possible future ideas which most probably would not be implemented in the current scope of the thesis.

4.1 Important Terms

Below is the list of important terms that appear throughout the requirements sections and further chapters:

- **Anonymous User** – A User who is not yet registered in the system or has not logged in.
- **User** – An object containing information such as email, display name and other User parameters.
- **User Profile** – A Frontend-only abstraction which shows Base User information.
- **Artist** - A User with additional privileges, for example, Song or Collection creation.
- **Follower/Followee** - A relationship between a User and another User where one subscribes to another.
- **Friend** - A Followee and Follower who are mutually subscribed to each other.
- **Song** – An object representing a processed audio file with additional metadata such as title or creation date.

- **Collection** – A container that holds multiple Songs.
- **Playlist** – A Collection consisting of pre-existing (already uploaded) Songs.
- **Album** – A Collection consisting of newly uploaded Songs.
- **Playback** – A process during which the User is receiving or manipulating audio data.
- **Playback Item** – A Song or a Collection.
- **Playback Queue** – A User-specific list of Playback Items which determines the playing Song order.
- **Liked Songs** – A private, system-managed Collection that holds the Songs a User has marked as liked.
- **History** – A private, system-managed Collection that records the Songs a User has played.
- **Playback Device** – A device on which the app is currently used.
- **Active Playback Device** – The User's Playback Device that is currently producing audio. All other Playback Devices are passive and mirror the Active Device's Playback position.
- **Friend Activity** – A display of the Songs currently listened to by the User's Friends.
- **Publication** – Textual User input.
- **Attachment** - Non-textual User input. For example, a Song attached to a Publication.
- **Reply** – The same as Publication, but created in relation to another, 'parent' Publication.
- **Feed** - A list of Publications. Either User-specific or Global (from multiple Users).
- **Chat** - A list of Publications specific to two or more Users.

4.2 Use Cases

A use case describes what an actor(User) can do with the system.

While the requirements in the following sections list what the system must do, use cases show the same functionality from the actor's perspective. This can make further modeling simpler, since the minimal subset of actions any given User can take is clearly defined.

The use cases are split into six groups: Account and Identity, Social, Library and Content, Playback, Search, and Communication. Each group has its own use case diagram. The Artist actor inherits from the User actor, so every use case available to a User is also available to an Artist.

The use case diagrams for each group are collected in the Diagram Catalog appendix (Figures C.1 through C.6).

4.3 Functional Requirements

Must Have

- The system shall support User and Artist models.
- The system shall allow Anonymous Users to create a User model and log in to the system.
- The system shall allow Users to change model information (name, bio, avatar, etc.) after the creation.
- The system shall support over-the-net audio Playback.
- The system shall provide a way to control the audio Playback.
- The system shall provide a way to display Playback Items uploaded to it.
- The system shall provide each User with a library view of their owned/liked content.
- The system shall limit content presentation based on a specific User. That includes Playback Items, User Profile, User Playback Devices, and other related items.

- The system shall support the creation of Playback Items. Playlists shall be created by all Users. Albums shall be created by Artists only.
- The system shall provide a Playback Queue that the User can modify.
- The system shall allow Users to mark individual Songs and entire Collections as liked.
- The system shall provide Chat capabilities, including direct Chats between two Users and group Chats with multiple members.
- The system shall allow any Publication to be created as a Reply to another Publication.
- The system shall allow Users to add Publications (Comments) under public Collections.
- The system shall provide a personal Feed of a single User's Publications and a global Feed that aggregates Publications across Users.
- The system shall allow Users to follow and unfollow other Users. Mutual follows shall be treated as a Friend relation.
- The system shall have a way to search Playback Items, Users, and Artists.
- The system shall display Friend Activity, that is, the Songs currently being played by the User's Friends.

Should Have

- The system shall support adding Attachments, such as Songs, Collections, or Users, to Publications and Chat messages.
- The system shall provide a notification system.
- The system shall make it possible to filter content by by the viewer's relation to the author.

- The system shall allow a User to the Active Playback Device between them.

Could Have

- The system shall make it possible for multiple Users to listen to Songs in a synchronized manner.
- The system shall provide an Artist interface that allows Artists to see statistics related to their content.
- The system shall have a forum section.
- The system shall provide content recommendations based on a User's listening history

4.4 Non-functional Requirements

Must Have

- The system shall provide access to its contents only to authorized Users. The only exceptions shall be the "Log In" and "Sign Up" pages.
- The system shall make unauthorized access to data as complex as possible, so that even in the case of misconfiguration of access rights, the data are hard to get to.
- The system shall store uploaded content securely.
- The system shall store and transport sensitive User information safely. I.e., it should use safe transport protocols, such as HTTPS and WSS .
- The system should be resistant to the most common web-based attacks: CSRF, XSS, etc.
- The system shall, in case of unexpected failure, remain rendered on the screen and show the appropriate non-technical message to the User.

- The system shall minimize the network load it has on the server and the User device. That includes reducing the data footprint and implementing eager loading strategies.

Should Have

- The system shall provide a responsive search method that reacts dynamically to User input.
- The system shall keep Playback position and the current Song synchronized across a User's open Playback Devices.

Could Have

- The system shall support offline Playback.
- The system should integrate a DRM mechanism.

The next chapter presents the high-level technology choices used for the implementation.

5 Implementation Planning

Before the actual implementation could begin, it was necessary to choose the languages, frameworks, and other technologies that would shape the overall architecture of the application. As mentioned before, the goal of the thesis is a web application, hence, this chapter focuses on the technologies mostly suitable for web applications. This chapter discusses the following areas: Audio Processing and Playback, Authentication, Server–Client Communication, Backend, and Frontend.

5.1 Audio Processing and Playback

Audio Processing and Playback are the most important aspects of the application and were researched first. There are four main approaches used today: Basic Download, Progressive Download, Streaming, and Peer-to-Peer.

- **Basic Download**

With Basic Download, the entire file must arrive before Playback can start. The client issues a standard HTTP GET, and the server returns the media as raw binary data in the response body. [19] For high-quality files this means significant storage space and download time, and once a download has started the system cannot adapt to a change in network conditions.

- **Progressive Download**¹

Progressive Download lets Playback start before the full file arrives. The client fetches a short initial part over HTTP, begins Playback, and requests further parts as needed, usually via HTTP byte-range requests. [21] The player relies on audio container metadata (e.g., the header of an MP4 file) that maps playback time to byte offsets, so it can request the next few seconds of audio or seek to a different position. [22]

Storage becomes less of an issue than with Basic Download, since the player only needs a small in-memory buffer and older

1. Sometimes called ‘Pseudo-Streaming’. [20]

data can be discarded once played. Adaptive playback is also possible, but it is not standardized. It requires custom preprocessing of the audio on the server and custom client logic to choose between representations during Playback.

- **Segmented HTTP Streaming**

Segmented HTTP Streaming bakes adaptive playback into the protocol itself, rather than leaving it to the application. The audio file is split into many small chunks, and a manifest is generated that lists the chunks and describes which time range each chunk covers (usually a couple of seconds). [23] The most common implementations of this approach are MPEG-DASH and HLS, both of which are described in **Chapter 6**. The client downloads individual chunks instead of working with byte ranges in a single large file, and the manifest provides the chunk-to-time mapping needed for Playback and seeking.

The original audio file can also be encoded into multiple representations of different sizes and qualities, all referenced by the same manifest. The client picks chunks based on network conditions or device capabilities, and since each chunk is a separate HTTP resource, retries and caching are handled efficiently by standard HTTP caches and CDNs. [24]

- **Peer-to-Peer**

With Peer-to-Peer transfer, audio data is shared directly between users' devices rather than served from a central server. [25] This can reduce server load, but Playback speed depends on peer availability and the network conditions on the other end.

Streaming was chosen as the playback method for this thesis.

Basic Download was excluded because it requires downloading the whole file before playback can start, making the usability of the application limited. Progressive Download improves startup time and seeking, but adapting to changing network conditions is not standardized and would require custom preprocessing of audio files on the server side, as well as writing client logic that could utilize this approach. Peer-to-Peer transfer was also excluded, since it would rely on users' devices as sources of audio data, making playback speed dependent on peer availability and location.

Segmented HTTP Streaming addresses these issues by defining adaptive playback directly in the manifest and delivering the audio as small chunk files. Additionally, its integration into the service, as will be demonstrated later in **Chapter 6**, is straightforward.

The next key choice would be the authentication method, as it can significantly influence both the design of the Frontend and Backend.

5.2 Authentication

There are many different methods of authentication available, but only those suitable for web applications are considered: Basic Authentication, Session-Based Authentication, Token Authentication, and Third-Party Authentication (OAuth2).

- **Basic Authentication**

With Basic Authentication, what is sent on every request is the User's actual password, Base64-encoded for transport but not encrypted in any way. The credentials are placed in the HTTP Authorization header, and the browser typically caches them after the first successful login and attaches the header to subsequent requests on its own. [26]

If this value leaks, for instance into a log or telemetry record after HTTPS is terminated at an internal proxy [27], what leaks is the password itself rather than an opaque token. Revocation therefore requires a password change. There is also no protocol-level notion of *logout* or per-device sessions, so signing a User out requires Frontend-side workarounds.

- **Cookie-Based Session Authentication**

Session authentication keeps the credential on the server and only sends an opaque identifier to the client. The user logs in with their credentials, the server creates a *session* record holding the relevant user information (ID, roles, and so on), and instructs the browser to set a cookie with the session ID. [28] This ID is sent on subsequent requests instead of the credentials and identifies the logged-in user.

Compared to Basic Authentication, this enables explicit server-side logout and revocation (by deleting or invalidating sessions),

as well as per-session policies such as device tracking and inactivity timeouts. The main trade-off is the need for a session store, typically a relational database or an in-memory key-value store, which adds infrastructure and a potential bottleneck if not configured correctly.

- **Token Authentication**

Token Authentication replaces raw credentials with a token issued by the server after login, which the client sends in the Authorization header under the Bearer identifier. For a single-page application, the trade-off that matters most is where the token lives in the browser: if held in `localStorage`, it is reachable from any script on the page, so a Cross-Site Scripting (XSS) injection can read it, while session cookies marked `HttpOnly` are not exposed to scripts.

Tokens come in two common variants. A *database-backed token* is stored on the server alongside the user record and looked up on every request, similar to a session but carried in a header instead of a cookie. A *self-contained token*, such as a JSON Web Token (JWT), carries the user identity and permissions in its payload and is verified by signature, with no lookup. Self-contained tokens simplify horizontal scaling, but revocation is hard. Once issued, a token stays valid until it expires, unless a revocation list is kept, which brings back state and coordination. In both variants, the Frontend has to attach the token to every request and handle refresh explicitly, whereas cookies are sent by the browser automatically.

- **Third-Party Authentication (OAuth2)**

Third-Party Authentication is a complement to one of the methods above rather than a replacement for it. It delegates login to an external identity provider via the OAuth 2.0 protocol, such as Google or GitHub. [29] The application redirects the user to the provider, receives an authorization code in the callback, and exchanges it for tokens that identify the user. Password storage, multi-factor authentication, and account recovery are then handed off to the provider. However, this cannot be the only authentication method an application offers, since it would

require every user to have an account with one of the supported providers.

Based on the characteristics of the authentication methods, it was decided to use the Session Authentication.

Compared to Basic Authentication, it avoids sending user credentials with every request, supports explicit logout, per-session security policies, and extra session-related data. Compared to Token Authentication, sessions offer simpler revocation, do not require managing tokens, and are sufficient for a web application that does not need stateless or cross-service authentication. Third-Party Authentication based on OAuth 2.0 can be added on top of this design.

Another important concept that must be addressed is the server-client communication.

5.3 Server-Client Communication

Some of the features outlined in **Chapter 4** (such as playback handling, and immediate content updates) require bi-directional communication. Hence, a method for communication from the server to the client must be added.

Because this is a web application, techniques such as Webhooks, gRPC, Pub/Sub, and others that are not directly supported by browsers are not discussed. Traditional Polling is also not considered due to its performance limitations and inefficiency. P2P techniques (e.g., WebRTC) are not listed because the application's architecture is client-server, not peer-to-peer.

This thesis focuses on four commonly used methods for server-client communication: Long Polling, Server-Sent Events, WebSockets, and Push API.

- **Long Polling**

Long Polling simulates server-to-client communication by holding an HTTP request open until the server has something to send, after which the client immediately issues another request. [30] It works in environments where server messages are rare, but for high-throughput scenarios with frequent small payloads, it adds significant overhead from repeatedly opening and closing HTTP connections. [31]

- **Server-Sent Events (SSE)**

SSE is a one-way protocol that lets the server push data to the client over a single long-lived HTTP connection. The client opens the connection with an HTTP request, and the server keeps it open and uses it to send any number of subsequent events. [32] Compared to Long Polling, no new connection is needed per update, while still running over plain HTTP, which fits most networking setups.

The trade-off is that the proxy and the server must track all connected clients, and the Backend must add routing logic to deliver each event to the right connection. [33]

- **WebSockets**

WebSockets provide full-duplex communication over a single TCP connection, established by upgrading a standard HTTP connection via the WebSocket Protocol Handshake. [34] Messages can flow in both directions at any moment, which suits applications that need frequent two-way communication. [31]

As with SSE, adding WebSockets requires extra infrastructure and logic for connection and message routing. In addition, WebSocket messages do not follow the HTTP request-response model, so they cannot be cached by standard HTTP caches [35] or CDNs, and they do not benefit from built-in features such as idempotent retries.

- **Push API**

The Push API is intended for browser notifications, not for interactive state updates. [36] The server sends messages to a per-domain push endpoint, the browser queues them for a background *service worker*, and the browser is free to throttle that worker depending on system load. [37, 38] The resulting latency is higher and less predictable than with direct protocols such as WebSockets, making the Push API unsuitable for time-sensitive updates such as Playback state. [36]

Based on the requirements outlined in **Chapter 4**, at least three features would require server-to-client communication:

- **Playback state synchronization.** Any Playback Device the User acts on has to send the change to the server, whether it is a play, pause, seek, or song switch, and the Active Playback Device additionally sends periodic position updates to correct drift. Even at a moderate rate, these events add up, and each one would otherwise need its own HTTP request. The server then forwards the new Playback state to the User's other Playback Devices to keep them in sync.
- **Real-time typing indicators in Chats and Comment threads.** The client would send multiple short updates to the server while the User is typing. The server would then broadcast these events to the other participants, so that the current typing activity is displayed in real-time.
- **Live Chat messages and Comments.** When another User sends a message in an open Chat, or posts a Comment under a Collection that the User is currently viewing, the new content should appear on the screen immediately, without periodic polling from the client.

The initial implementation used SSE. The read-path features in the list above (Live Chat messages, Comments, typing indicators) are mostly server-to-client and would have been covered by a long-lived SSE stream for the receive path with ordinary HTTP requests for the send path. Playback synchronization, however, does not fit this split well. When the User drags the position slider quickly, the primary Playback Device sends a short burst of position updates to the Backend, which then forwards them to the secondary Devices. With SSE for the receive path and one HTTP POST per update on the send path, every update in that burst is a separate HTTP request. Even with a single User this adds extra lag, and with many Users active at the same time it puts real load on the server.

Additionally, mature WebSocket support is widely available in modern Python web frameworks, often as a built-in feature with primitives such as named groups and per-user broadcast. Equivalent SSE support is less standardized and typically requires a separate pub/sub layer.

For these reasons, WebSockets are used as the primary mechanism for real-time communication. The drawbacks listed above are accepted in exchange for a uniform bidirectional channel and lower integration cost with the chosen Backend framework.

These choices define what the Backend and Frontend have to support, and Backend is discussed next.

5.4 Backend

5.4.1 Language

Python was chosen as the language for the Backend of the application. It is well-suited for web development, since it provides many libraries and built-in interfaces for HTTP APIs, database access, background processing, and other areas used in the implementation. Its syntax also allows fast prototyping, which is helpful while iterating on the data model and the overall app architecture. Performance is good enough for the scale of this project, as the application is not expected to handle very high request rates, and the most computationally heavy task, audio processing, is handed off to external CLI tools (see **Chapter 6**).

5.4.2 Python Framework Comparison

There are currently three widely used frameworks available for Python: FastAPI, Django, and Flask.

- **FastAPI**

As the official website states:

‘FastAPI is a modern, fast (high-performance), web framework for building APIs with Python, based on standard Python type hints.’ [39].

Its main emphasis is compatibility with Python’s `asyncio`, which makes it much easier to write asynchronous code in comparison to the other two frameworks. In addition, it integrates well with Pydantic [40] making user input validation straightforward. Moreover, routing/middleware is trivial to configure. [41]

However, FastAPI is relatively new and simple. It can work well for small or highly custom projects, but for standard setups, it lacks many convenience wrappers. For example, the WebSockets integration does not provide connection lifecycle management or message broadcast capabilities. [42] Additionally, no pre-defined templates are available for basic CRUD operations, so they must be implemented manually, which adds unnecessary complexity.

- **Django**

Django is a “batteries-included” [43] framework in the sense that it has tools for most of the common web development tasks. It provides built-in user session management, easy CSRF and XSS protection setup, built-in file storage abstractions, and integrated handling of media files (images, audio, etc.). Support for REST APIs can be added through `django-rest-framework` [44], which provides generic views, serializers, and routers that make the implementation of CRUD endpoints quick with minimal custom code on top. Real-time functionality can be implemented using `django-channels` [33], an official Django package that adds connection lifecycle management, group-based message broadcasting, and integration with Django’s authentication and session systems. As the documentation states, one of the main advantages of this framework is the time savings it offers. [45]

Nonetheless, some notable drawbacks include its opinionated design choices, which can make it hard to add custom logic. Furthermore, the size of the resulting codebase can be larger because the framework ships with many modules that may remain unused, while still occupying storage space (for example, RSS integration or server-side rendering modules).

Finally, the most significant disadvantage is its limited support for asynchronous processing [46]: plenty of ORM operations (e.g., transactions), database connection management, and many third-party middlewares/libraries remain synchronous / `async-unsafe`.

- **Flask**

Flask is a lightweight Python web framework that provides

only a small core with routing, request/response handling, and templating, while most other functionality is added through extensions. [47] This modular design is suitable mainly for small services or prototypes with a limited and clearly defined scope.

However, for typical JSON-based APIs, FastAPI offers similar functionality with fewer dependencies and less setup, while Flask usually requires several additional libraries and manual configuration to achieve the same result. At the same time, Flask, like Django, is based on the synchronous WSGI interface. WebSocket support is typically added via additional tools, such as SocketIO [48], which requires the Frontend to use the same library instead of the standard WebSocket API, introducing unnecessary coupling. As a result, Flask effectively combines the disadvantages of the other two frameworks: it lacks Django's integrated ecosystem and does not provide FastAPI's modern asynchronous features. In the context of this application, it does not bring any clear advantage over Django or FastAPI.

Summing up, of the three frameworks, Django is the most suitable one. For a music streaming platform, its media handling capabilities, WebSocket support via `django-channels`, and CRUD endpoint abstractions of `django-rest-framework` lead to a better developer experience, shortening the overall implementation time. The limited support for asynchronous code is also less of a problem in practice for this project, since WebSocket handling already requires Django to run under ASGI through `daphne` (which `django-channels` depends on). For the synchronous request paths, Instagram has shown that Django can still handle large loads. [49]

Having established the Backend framework, the next step was to choose the database.

5.4.3 Database

The Django framework supports several relational databases: SQLite, PostgreSQL, MySQL/MariaDB, and Oracle. Oracle is excluded due to its commercial license [50]. Otherwise, for a project of this size, any of these options would be fast enough, because the expected number

of users and the amount of stored data are not significant. Therefore, performance is not the primary factor in choosing a database.

Instead, the choice is based on the available features and how well they are integrated with Django. In this application, two requirements are especially important: full-text search and UUID primary keys (see **Chapter 6** for details). And for these requirements, PostgreSQL is the most suitable option.

First, PostgreSQL offers good full-text search support and additional tools for fuzzy matching, such as trigram indexes. [51] As a result, it can be used as the base for full-text search instead of a separate service like Elastic or Sphinx. [52, 53] Django exposes these features through the `django.contrib.postgres` module, which simplifies the creation of full-text and trigram-based queries directly from Python code. [54, 55]

Second, PostgreSQL has native support for UUID values via a dedicated `uuid` column type. [56] This type stores UUIDs in a compact binary form and allows efficient indexing and comparison.² In many other databases UUIDs must instead be stored as plaintext (e.g., `CHAR(36)`), which is less space- and index-efficient. Django's `UUIDField` also maps cleanly to PostgreSQL's native type, so UUID-based identifiers can be used without extra configuration.

Lastly, with the database chosen, the Frontend application type, language, and framework had to be selected.

5.5 Frontend

5.5.1 Application Type

Modern web applications are typically built either around server-side rendering (SSR), where most of the HTML is generated on the server and then retrieved by the browser to be later rendered, or around client-side rendering (CSR), where most HTML is generated in the browser using JavaScript. A single-page application (SPA) is an application that keeps a single HTML document loaded and updates the user interface dynamically on the client, usually relying on CSR, sometimes combined with SSR for the initial page load.

2. If using time-based UUID types, e.g., UUIDv7 [57]

In an SSR setup, frameworks such as Django generate most of the user interface on the server and send complete HTML pages (or partial updates) to the browser, often only requiring small pieces of JavaScript for interaction. [58] In contrast, an SPA typically fetches JSON or other data from the server and, based on it, renders and updates the user interface directly in the browser. [59]

The research for the Frontend began by deciding which of these approaches to use. In this project, the application must maintain a lot of state directly in the browser: the current Playback position, the Playback Queue, the volume level, the Chat state, and so on. These values change frequently and in small steps, sometimes several times per second, and require immediate feedback for the user. Moreover, the user interface must respond to events sent by the server over WebSockets, such as playback updates or chat messages. A purely server-rendered approach with full page reloads or round-trips for each small change would be unsuitable for this kind of highly interactive, real-time interface and would complicate state handling across devices. Because of these requirements, the Frontend is implemented as a client-side rendered single-page application (SPA).

5.5.2 Language

Currently, there are many libraries and frameworks in different languages that can be used to write web interfaces. For example, it is possible to create them even in low-level languages, such as C [60] or Rust [61]. Nevertheless, JavaScript remains the industry standard because it is still the only general-purpose language directly supported by browsers. It means that JavaScript is the only language that can be executed natively by the browser's runtime, without the need for previous pre-processing (e.g., compilation to WebAssembly). This simplifies the development, build, and deployment process, and for this reason the client application is based on JavaScript.

The project, however, does not use plain JavaScript, but rather TypeScript on top of it. TypeScript is a typed superset of JavaScript that compiles to plain JavaScript, so the code that the browser actually runs is still JavaScript. The benefit comes from the additional type checking performed during the build step, which catches many simple errors before the code is run.

5.5.3 Framework

Modern Frontend development is mostly based on libraries or frameworks such as React, Vue, or Angular. [62] For this project, React [63] was chosen.

React is a library for building user interfaces from reusable components. Its core only handles rendering, so common needs such as routing, state management, data fetching, and UI components are added through mature ecosystem libraries instead of being written from scratch.

React provides a built-in mechanism for component state: when a value in that state changes, the parts of the interface that depend on it re-render automatically. This matches the requirements established earlier in this section, since values such as the Playback position, queue, volume, and live Chat messages change frequently and need to appear on screen immediately, without manual DOM updates. React is also widely adopted [64], so examples and solutions to common problems are easy to find.

5.6 Summary

The platform follows the Model–View–Controller architectural pattern [65] and is organized into three logical layers:

- **Model**

The Model layer is responsible for representing and manipulating the application data. In this project, it consists mainly of the Django models with entities such as Users, Songs and Collections; and the data processing logic, for example audio conversion or data querying.

- **View**

The View layer is what the user interacts with directly. It presents the data from the Model layer and turns user actions into requests that can be processed by the rest of the system. Here, the View is implemented as a single-page application in JavaScript using React. It renders the interface, displays information such as playback state or friends' activity, and sends user actions

(for example, play/pause commands or chat messages) to the Controller.

- **Controller**

The Controller layer connects the View with the Model. It receives HTTP and WebSocket requests from the Frontend, applies cookie-based session authentication and access control, and then calls the appropriate Model logic. In this platform, the Controller corresponds to the Django REST API and WebSocket endpoints.

In other words, the Django models and database layer form the Model, the React single-page application acts as the View, and the Django-based REST and WebSocket endpoints play the role of the Controller that coordinates communication between them.

The next chapter, **Chapter 6**, describes the concrete implementation of these parts and how they work together to form the complete music streaming platform.

6 Implementation

This chapter goes over the built application, explaining the choices behind each feature, and focuses on what is not clear from the diagrams or the source code itself. Structure and naming that the diagrams or code already show are not repeated in the text. The high-level system overview is in `text/diagrams/HighLevelSystem.png`, the Backend and Frontend architectures in `text/diagrams/BackendSystem.png` and `text/diagrams/FrontendSystem.png`.

6.1 Stack

The Backend runs Django 5.2 on Python 3.14, served by the Daphne ASGI server because `django-channels` needs ASGI for WebSocket support [66]. The HTTP API uses Django REST framework with `django-filter` [67], authentication uses `dj-rest-auth` [68] on top of `django-allauth`, and background work runs on Celery. Database is PostgreSQL 17, Redis [69] is used as both the cache and the data layer for `django-channels`. Dependencies are managed with `uv` [70].

The audio pipeline uses `FFmpeg` [71] for transcoding and `Shaka Packager` [72] for segmentation, both invoked as external CLI tools.

The Frontend is a Vite-built React 19 SPA written in TypeScript, styled with Tailwind and `shadcn/ui`. Queries and mutations are managed via TanStack Query and HTTP calls go through `axios`. Forms use `react-hook-form` with `zod` validation, and Playback uses `Shaka Player` [73] for DASH and HLS streaming.

Deployment is a simple Docker Compose behind a Caddy reverse proxy on a Hetzner VPS, with Mailgun for mail services and PorkBun for the domain.

Full dependency lists are located in `src/backend/pyproject.toml` and `src/frontend/musikk/package.json`.

6.2 Audio Processing and Streaming

This section follows an uploaded audio file through the pipeline that prepares it for Streaming. The Streaming Protocols are covered first, since the rest of the pipeline has to match what they allow.

This section stays at the level of what each stage does and what parameters it uses. The rationale behind the choices and the per-flag breakdown of every call are in the Audio Processing Pipeline Details appendix. Anything mentioned below but not explained in detail is covered there.

6.2.1 Streaming Protocols

Two protocols are used: MPEG-DASH [74] on devices that support Media Source Extensions, and HLS [75] on iOS, which does not support MSE. The basic principles behind both protocols were covered in the Segmented HTTP Streaming section.

Both protocols expect audio split into short chunks paired with a manifest. The Packaging step is the one that produces those.

6.2.2 Packaging

Packaging cuts the transcoded audio into short fragmented MP4 segments and generates a DASH manifest (*.mpd) and an HLS master playlist (*.m3u8) that point at them. [76] The segments follow Common Media Application Format (CMAF), so a single set of chunks works for both protocols. The output is one initialization segment (*.init.mp4) per representation and a sequence of media segments (*.m4s).

Packaging only arranges existing audio. The CLI tools that produce and analyze that audio in the first place are introduced next.

6.2.3 Processing Tools

Three CLI tools run the pipeline: FFmpeg [71] for Transcoding and audio analysis, FFprobe (shipped with FFmpeg) for reading source metadata, and Shaka Packager [72] for the Packaging step. FFmpeg can package too, but its output had compatibility issues with Shaka

Player [73], used on the Frontend for Playback, so a separate packager was chosen.

With FFmpeg in place as the encoder and analyzer, the Transcoding configuration it runs can be described.

6.2.4 Transcoding

Transcoding decodes the source audio and re-encodes it into the target codecs and bitrates. Raw audio formats such as WAV are too large for Streaming, for example a 3-minute stereo 24-bit 44.1 kHz recording is roughly 50 MB. Lossless FLAC and lossy AAC/HE-AACv2 were selected as the output codecs, since HLS only allows a small set of audio codecs. [77] The AAC encoder used is `libfdk_aac`, chosen over the other AAC encoders shipped with FFmpeg for its quality and licensing terms.

The output configuration was chosen based on existing research: [78, 79, 80, 81, 77, 82]

- **AAC:** 96, 160, and 320 kbps
- **HE-AACv2:** 24 kbps
- **FLAC:** lossless

Multiple bitrates support adaptive streaming. The AAC tiers use Constant Bitrate (CBR), so the Frontend's Adaptive Bitrate logic has a stable per-tier bandwidth to pick from.

Transcoding needs a clean input with reasonable parameters. The Audio Preprocessing step described next ensures that each upload is valid and within the proper limits, and also captures the metadata of the result.

6.2.5 Audio Preprocessing

Before Transcoding, the upload is validated and normalized. After Transcoding, the loudness is measured. The sub-blocks below cover input validation, metadata extraction, the normalization filter chain, and the loudness measurement.

Input validation

The upload passes through a chain of checks. The file-size and MIME checks run on the raw bytes. Other checks use values reported by FFprobe. Together they restrict the worst-case storage and Transcoding time of a single upload:

- **MIME type**, inspected with `python-magic` [83], must be `audio/*` and belong to an allow-list of supported subtypes.
- **Codec**, reported by FFprobe, must belong to an allow-list of stable codecs.
- **File size**: up to 2 GB, roughly a 2-hour 24-bit, 48 kHz stereo WAV.
- **Duration**: up to 7200 s (2 h).
- **Sample rate**: 1 Hz to 384 kHz.
- **Channel count**: 1 to 8.

The allow-lists are defined in `streaming/audio/config.py`.

Input metadata extraction

FFprobe reads the source codec, sample rate, channel count, bit depth, duration, and bitrate. These values are reused by the input validation above. They also narrow which outputs the Transcoding step produces: FLAC is skipped for lossy sources, and AAC tiers whose target bitrate is above the source bitrate are dropped.

Normalization and loudness

Three normalization filters run as a single FFmpeg `-af` chain applied inside every Transcoding subprocess. A separate loudness measurement then runs after Transcoding. The steps are:

- **Channel normalization**. Sources with three or more channels are downmixed to stereo. Mono and stereo inputs are kept as-is.

- **Sample rate normalization.** Each source is resampled to the base of its rate family (44.1 kHz or 48 kHz). Files below 44.1 kHz are kept as-is. Non-family rates default to 44.1 kHz.
- **Bit depth conversion.** All audio is converted to signed 16-bit samples, since `libfdk_aac` does not accept higher precision.
- **Loudness measurement.** After Transcoding, FFmpeg's `ebur128` filter produces an integrated value in Loudness Units relative to Full Scale (LUFS) and a true peak in dBTP. The measurement runs on the FLAC output if it was produced, otherwise on the first AAC output. Both values are stored on the Song and used by the Frontend player to apply a consistent Playback gain.

Once Transcoding and Packaging finish, the original upload and the intermediate transcoded MP4s are removed. Only the packaged segments, manifests, and the loudness values stored on the Song row survive.

The Backend code that runs all of these stages is described next.

6.2.6 Backend Pipeline

The Backend code is under `src/backend/musikk/streaming/audio/`, the class layout is in `text/diagrams/AudioClassDiagram.png`.

Pre-pipeline validation (the file-size, MIME, and FFprobe-based checks from the previous subsection) is defined in `validators.py` and runs before any audio processing is started. The pipeline itself, in `processing_pipeline.py`, holds a shared `ProcessingContext` which is used by `Step` instances, so it is easy to add new steps or change existing. The configured step chain runs `FFmpegStep`, then `Loudness-MeasurementStep`, then `ShakaPackagerStep`. Each `Step` hands off to a CLI command wrapper: `FFmpegStep` calls `FFMPEGFull`, and `ShakaPackagerStep` calls `ShakaPackagerMPDAndM3U8`. Each step has a `rollback` method. If a step raises, the previously executed steps are rolled back in reverse order. The rollback clears the per-run temporary directory and removes any output already written to Django storage.

`FFmpegStep` runs one FFmpeg subprocess per output tier in parallel via a `ThreadPoolExecutor` sized to the number of selected tiers.

Threads are used rather than processes because FFmpeg itself is multi-threaded inside each subprocess, so the Python side only waits on the subprocess to finish.

When this pipeline is started, and how its status reaches the Frontend, is covered in the Audio Upload section.

6.2.7 Frontend Playback

As mentioned in the Stack section, Shaka Player is used for Streaming on the Frontend. It is an open-source JavaScript player for DASH and HLS that reads the Streaming manifests, fetches segments over HTTP, and passes them into an HTML audio element, picking the audio tier that matches the available bandwidth. That audio element is owned by the `AudioPlayerController` described in the Playback and Device Handling section.

This concludes the audio pipeline. The next section covers the WebSocket layer.

6.3 WebSocket Communication

This section covers the WebSocket setup of the application. The Backend code is under `src/backend/musikk/websockets`, the Frontend code under `src/frontend/musikk/src/ws`. The end-to-end connection lifecycle is in `text/diagrams/WebsocketLifecycle.png`, and the class layout in `text/diagrams/WebsocketsClassDiagram.png`.

Two WS message-related terms are used throughout the text: *Events* are messages sent from the server to the client and *Actions* are messages sent from the client to the server.

6.3.1 Backend Handling

All real-time features share a single connection at `/ws/user`. With one connection per feature, additional message ordering logic would need to be implemented: for example, a Playback Device switch and a Playback activation actions could arrive in any order and it would not be clear which one to process first. In addition, each extra connection also adds memory and file-descriptor overhead on the server, can

run into the browser's connection limit, and needs its own reconnect handling on the Frontend.

As discussed in Server-Client Communication, WebSocket support is added via `django-channels`. The `BaseConsumer` class manages connections and is instantiated once per connection.

For broadcasts to multiple connected clients, `django-channels` provides groups. A group is a set of consumers instances (connections), and a message send to the group reaches every member.

Two kinds of groups are used. The User's own group `user_<uuid>`, which transmits "private" messages: Playback Device list, Notifications, Liked Songs change and other relevant events. A User with several tabs (devices) open uses one consumer per tab, and all of them are joined to the same `user_<uuid>` group, hence a "private" broadcast reaches every tab. Named topic groups (`topic.<prefix>.<id>`) can be joined by several different Users at once and are used for shared messages: Chat messages, Collection Comments, and typing indicators.

Actions are handled by `WSActionHandler` subclasses. Each handler defines a maps of action names to the methods that run them. The consumer then dispatches every incoming action to the handler method registered for its name.

Events are sent through two helpers, `send_ws_event` for a group and `send_ws_event_to_channel` for a single channel, so any part of the Backend can emit them. DRF views, Celery tasks, and Django signal handlers can call these helpers directly or use the special Broadcaster classes introduced in

When the connection closes, each handler runs an `on_disconnect` hook so per-feature state is released: joined topic groups are dropped, the registered Playback Device is cleared, and so on.

A Playback event can also include an excluded device id, in which case the consumer skips delivery to the device with the specified id. This is useful, since the device that sends an update already is in the updated state and does not need a WS event with the new data from the server.

6.3.2 Frontend Handling

Frontend opens a single connection to `/ws/user`. Authentication is implicit, the browser sends the session cookie along with the upgrade request. The WS-handling code is split into two classes, `WSClient` and `WebSocketManager`.

Reconnect logic is implemented, so a tab that is put to the background or a brief network outage does not require a page refresh or manual actions from the user. Reconnects use exponential backoff with a 30 s ceiling and random jitter. The jitter prevents many tabs from reconnecting at the same time after an outage.

`WSClient` wraps the native `WebSocket` interface. Most actions go through the `send` method, which queues them and flushes the queue when the connection is open. An action issued while the connection is down is therefore not lost, and the order of queued actions is preserved on flush. The exception is the periodic Playback position sync action, it is sent only if the connection is open. Otherwise, for example, stale seek actions would be replayed on reconnect, even if the Playback position has already moved and stabilized.

Named topic subscriptions are handled via the `WSClient` as well. Subscriptions are reference-counted: only the first topic subscriber runs a `subscribe` action, and only the last unsubsubscriber runs the `unsubscribe` action. Both actions also bypass the queue. On reconnect, the on-open handler reads the `refcount` map and sends a `subscribe` for every topic in it. If `subscribe` also used the queue, the server would get the same `subscribe` twice. If `unsubscribe` used the queue, a `subscribe` and an `unsubscribe` sent while the connection was down would still reach the server on reconnect, even though the Frontend no longer needs the topic.

`WebSocketManager` owns the single `WSClient` via a `refcount`. `connect` returns the existing client and increments the counter. `disconnect` closes the connection only when the counter reaches zero. Without the `refcount`, one subscriber unmounting (including the React's `StrictMode` double-mount in development) would tear down the connection shared with the others, drop their topic subscriptions, and force a fresh handshake plus re-subscription on the next mount.

Feature code does not touch either class directly. Instead it uses hooks (e.g., `useChatWsEvents`) that subscribe to a topic and register listeners for the matching events.

WebSocket connection is also used for Frontend cache invalidation. The tab which changes the data invalidates the affected React Query key on mutation success, and every other open tab does the same when the server event signifying that new data is available arrives.

6.4 User Management

This section covers how Users are modelled, how the Frontend renders them, and the API that connects the two. The Backend code is under `src/backend/musikk/users`, the Frontend code under `src/frontend/musikk/src/features/user`. The class diagram is in `text/diagrams/UsersClassDiagram.png`, and the Frontend component hierarchy in `text/diagrams/FrontendUser.png`.

6.4.1 User Data Model

A single `BaseUser` model covers every account, with a `role` discriminator field that is either “Base” or “Artist”. An Artist is not a separate subclass, only a role value, since there are currently no Artist-only fields (a separate database table is not needed).

Each User is uniquely identified by their email. The `display_name` is not unique and defaults to a random string, so a new account always has a usable name before the User picks one.

User-to-User relations are modelled by a single `UserFollow` table. Friendship is not a separate model. A Friend is inferred from a mutual follow, two `UserFollow` rows in opposite directions. The `BaseUser.friends` property computes that join directly in the ORM, so the Friend set is never stored.

6.4.2 Backend

The endpoints are listed in `src/backend/musikk/users/api/v1/urls.py`.

The `/users/me` endpoint splits the serializer by HTTP method. GET returns the public fields plus the User’s own email. PATCH ex-

poses only `display_name`, `bio`, and `avatar`. The split exists so that the `role`, `email`, and `is_artist` flag cannot be written through the same endpoint as profile edits, even if extra fields are sent in the payload. The public retrieve endpoint called for another User omits the email, so an email address only ever reaches the owner of the account.

A follow does more than insert a row. The POST handler is idempotent through `get_or_create`, so a double-click does not create a duplicate notification.

A successful follow also creates a `FollowerNotification` that appears in the target's Notifications. Unfollow is a DELETE on the same path. It broadcasts `USER_UNFOLLOWED` to both groups and removes the matching `FollowerNotification`, so the target's Notification window does not keep an invalid entry.

Three list endpoints under `/users/<uuid>/` (`friends`, `followers`, `followed`) are gated by the `IsSelfOrFriend` permission, so the connection lists are visible only to the owner or to one of their Friends (the social relations of a User are not considered public).

6.4.3 Frontend

`ProfilePage` covers both the own-profile and other-User views with one component, switching between an "Edit Profile" dialog and a "Follow/Unfollow" button based on whose UUID is in the route. The page layout is otherwise identical between the two views, so a split into two components would mostly mean keeping two copies in sync.

6.5 Account Creation and Authentication

This section covers how a new account is created, how the email address is verified, and how the resulting Session is established and maintained. The Backend code is under `src/backend/musikk/users` and the Frontend code under `src/frontend/musikk/src/features/auth`. The end-to-end flow across sign-up, email confirmation, login, and session expiry is in `text/diagrams/AuthFlow.png`. The Frontend component hierarchy is in `text/diagrams/FrontendAuth.png`.

6.5.1 Backend

The Backend does not implement its own authentication stack, since session handling, password validation, and email verification are security-sensitive and easy to get wrong. [84] Instead authentication is built using `django-allauth` and REST endpoints on top of `allauth` are provided by `django-rest-auth` [68] (located in `/api/v1/auth/`).

`allauth` is configured to identify Users by email, with `ACCOUNT_LOGIN_METHODS = ["email"]` and the username field disabled. Sign-up therefore takes only email, password, and a password confirmation. `BaseRegisterSerializer` adds a write-only `is_artist` boolean and sets the new User's role to "Artist" when the flag is true, so the role choice is set during sign-up rather than requiring a separate upgrade step.

A custom `BaseAccountAdapter` class overrides the default authentication flow. `clean_email` rejects an email that already exists case-insensitively, since the stock `allauth` check is case-sensitive and would let `user@x.com` and `User@X.com` register as separate accounts. RFC 5321 permits the local part of an address to be case-sensitive, but states that relying on it is a bad practice. In addition, `get_email_confirmation_url` returns a Frontend URL instead of a predefined HTML template.

Email verification is set to mandatory, i.e., a new account exists in the database immediately after registration, but login is blocked until the link in the verification email has been opened.

Sessions use the `cached_db` backend. Each session is written to the database and also kept in Redis, so lookups for currently active clients avoid a database query.

6.5.2 Frontend

Two axios clients are configured in `src/frontend/musikk/src/api/axiosConf.ts`. The split exists because the authenticated client defines a response processing step that triggers a global logout on 401, which would loop on the login request. Both clients share a request interceptor that copies the `csrftoken` cookie into the `X-CSRFToken` header on unsafe methods.

The 401 handler on the authenticated client uses a shared `authRef.logout` reference rather than calling `logout` directly. The reference is set inside an effect in `AuthProvider` and cleared on `unmount`. This keeps the `axios` module free of React imports and avoids a full-page redirect, which would remount the app and lose any operations that are currently being processed before the logout can finish.

The next section covers the main objects Users interact with: Songs and Collections.

6.6 Songs and Collections

This section covers the Song/Collection models and their Author relations. The Backend code is located under `src/backend/musikk/streaming`. The Frontend code is split across the `songs/` and `collections/` subdirectories under `src/frontend/musikk/src/features/`. The class diagram is in `text/diagrams/StreamingClassDiagram.png`.

The audio pipeline fields on Song and the Album upload flow are covered in the Audio Upload section. Playback Queue design is covered in Song Queue and Playback logic in Playback and Device Handling.

6.6.1 Data Model

`BaseSong` represents the literal Song object, i.e., the fields that make each song playable and identifiable: title, description, image, draft flag and the paths to the streaming manifests.

The app has only one Song container, the `Collection` class. It has four kinds, `Playlist`, `Album`, `History`, and `Liked Songs`, which are discriminated by a string field, the same pattern as the `role` field on `BaseUser`. This makes it possible to reuse one set of retrieve, list, and ordering operations, and only one Collection page component on the Frontend that is reused for all four kinds.

`CollectionSong` pairs a `BaseSong` with a `Collection` at an integer position. It also serves as the public identifier of a Song for most of the API endpoints. E.g., `/songs/<uuid>` resolves to a `CollectionSong`, not a `BaseSong`. The reason is that most actions on a Song target its slot in a specific Collection, not the Song in the abstract. “Remove

from this Playlist”, “jump to the Album”, render play button state, etc. A URL that accepts BaseSong would have to either also include the Collection uuid in every call, or do a lookup to find the row.

Authorship is stored in two join tables, SongCredit and CollectionCredit. SongCredit restricts the author to Users with the Artist role, while CollectionCredit accepts any User, since Playlists can be created by regular Users as well.

6.6.2 Backend

The endpoints are listed in `src/backend/musikk/streaming/api/v1/urls.py`.

The Collection manager exposes the queries that filter on the type discriminator: `.playlists()`, `.albums()`, `.histories()`, `.liked()`, plus `.public()` and `.published()`. List views such as User Library, Album by Song, and the Liked Collections view use these helpers, so the type filter is not repeated in each view.

Liking a Song and adding a Song to a Playlist both create a CollectionSong row, however they are exposed as two separate endpoints, `/liked-songs/add-song/<uuid>` and `/collections/<uuid>/songs`. The liked-songs Collection is always the User’s own, so the URL needs only the Song uuid. A Playlist add takes any of the User’s Playlists in the URL and requires the User to own it. The split is mainly there to keep the Frontend cache handling simple. The like endpoint emits a dedicated `LIKED_SONGS_CHANGED` event, which invalidates a single Frontend query that holds the set of liked Song uuids. Every like button reads its state from that set, so one update re-renders every instance of button on the page. Routing the operation through the generic Playlist endpoint would leave like state as a per-row `is_liked` flag, and the only invalidation signal would be the generic `COLLECTION_CHANGED` event (this was the original design). The same CollectionSong shown in an Album view and the Playback queue would then have two stale flags until both queries were updated.

6.6.3 Frontend

There are two main components which are reused throughout the application: SongCard and CollectionCard. Each takes the item to

render plus an optional `onClick` callback, a size variant, and a custom class name, so that they can be cleanly added to any other component.

6.7 Audio Upload/Album Creation

This section covers the design choices behind the Audio Upload. The end-to-end sequence is in `text/diagrams/AudioUploadFlow.png` and the Frontend component hierarchy in `text/diagrams/FrontendAudioUpload.png`. The Backend view `CollectionSongCreateView` is at `src/backend/musikk/streaming/api/v1/views/collections.py`. The Celery task `convert_audio` is at `src/backend/musikk/streaming/audio/tasks.py`. The Frontend code is under `src/frontend/musikk/src/features/album-upload`.

Audio uploads are modeled as asynchronous jobs because transcoding a long file can take minutes. Holding the HTTP request open for that long would give Frontend no way to show progress and time out behind a reverse proxy. Hence, the view validates the file, writes it to a temporary path, creates the Song row with `draft=true`, enqueues a Celery task that does the heavy processing “in the background”, and quickly returns 202 Accepted with the new Song UUID. Frontend generates an *operation id* per file and sends it in each Song upload request. The Celery task then includes the operation id in every WebSocket event it emits, so Frontend can update the right Song upload field even when several uploads from the same User run in parallel.

The view writes the file to a temporary path instead of passing its bytes to the task. Celery serializes task arguments through Redis, which keeps every queued message in RAM. A long lossless file would therefore sit in the broker until the worker picks it up, and a few concurrent uploads could easily fill up broker’s memory. The view passes only the path to the task, which both processes can reach.

Both Frontend and Backend validate the audio file. The Frontend checks file size and MIME type before the request, to reject obviously invalid files early. The Backend runs the same checks in `validate_audio`, plus duration, sample rate, channels, and codec, before enqueueing the task. The Frontend check is a UX shortcut, not a trust boundary. Any client can call the API directly, so the Backend check is the one that actually decides if the file is accepted.

A Song that is still being processed has no audio data yet, so it cannot be played. The Song is therefore created with `draft=true`, and the flag is cleared only after the Celery task writes the final audio fields, in one database transaction. Public list and retrieve endpoints filter on `draft=false`, so a Song that is still processing, or that has failed, is invisible to every listing query. Without the flag, every public endpoint would need its own status check, and a “half-built” Song could leak into search results. The Album has the same flag, and the Artist publishes the Album (which sets the flag to false) with a separate PATCH once every Song is processed.

Cleanup of the Song row is split from the Celery task. The task removes the files it created on failure, but leaves the row in place, because the Artist may retry a failed Song or leave the page mid-upload and come back. If the task deleted the row on failure, the Album the Artist is still editing would lose its Songs. A separate management command, `prune_draft_albums`, removes Album rows that have stayed in draft longer than a configurable window, together with their Songs and segments (can be ran via cron).

This concludes the upload path. The next section covers the Song Queue, the structure the User selects Playback Items from.

6.8 Song Queue

This section covers how the User can enqueue playable items in the application. The model code is under `src/backend/musikk/streaming/models/song_queue.py`, the Frontend component hierarchy is in `text/diagrams/FrontendSongQueue.png`, and the Backend sequence for adding a Collection and advancing is in `text/diagrams/SongQueue.png`.

6.8.1 Data structure

The Queue supports several frequent operations: insert at a random index, reorder, remove an item, pop from the front, pop several items up to a certain one, bulk append multiple items, and read the upcoming items in order. A straightforward doubly linked list in SQL from the first draft of the implementation did not work well for most of these.

Every insert, reorder, or remove cascaded pointer updates, and reading items in order needed linear traversal rather than a single range scan. A decimal index approach was chosen based on online discussions on this topic [85, 86]. The current model stores each `QueueItem` with a `Decimal` position and inserts a new item at the midpoint between its neighbours. A single insertion is a single DB write and reads are a simple range query.

The trade-off is that the position value can run out of precision when the User keeps inserting at the same spot. The position column is `Decimal(24, 12)`, so it has 12 fractional digits. This is enough for about 30 midpoint inserts between the same two neighbours before no number is left to put between them. However, in practice, the User does not edit the Queue that deeply most of the time, so this limit is fine for now. Appends use a separate `next_item_position` counter instead. The counter grows by a fixed gap (`POSITION_GAP = 1000`), so appended items never land on the same position and never use up the fractional digits.

6.8.2 Queue and PlaybackContext

The Queue itself is split into two classes. `SongQueue` holds items the User added explicitly, ordered by position. `PlaybackContext` tracks the Collection that is currently playing through a cursor into `CollectionSong.position` and a list of skipped indices. On advance, the Queue is consumed first, then the Context. If no items are left, Playback stops. The split lets the User add items on top of a playing Collection, work through them, and then return to where the Collection was.

6.8.3 QueueSource and lazy filling

Bulk-appending a large Collection does not materialize all items up front. A `QueueSource` stores either a single `Song` or a `Collection` with a cursor, and the `_fill` method converts Sources into Items in batches when the materialized item count drops below `REFILL_THRESHOLD = 30`. Each fill creates up to `FILL_BATCH = 20` items. This keeps the Queue table small even when the User queues an entire Album with hundreds of tracks, and it lets the Frontend show a stable window of upcoming items without paging through thousands of rows.

6.9 Playback and Device Handling

This section covers how Playback state is kept in sync across a User's tabs and Devices. The Backend code is under `src/backend/musikk/streaming`, with WS action handlers in `src/backend/musikk/streaming/ws/action_handlers/playback.py` and the per-User state stored in Redis via `PlaybackManager`. The Frontend code is under `src/frontend/musikk/src/features/player` and `src/frontend/musikk/src/features/playback`. The Backend models are in `text/diagrams/PlaybackClassDiagram.png`, the runtime managers, controllers, and broadcasters in `text/diagrams/StreamingWsClassDiagram.png`, the Frontend component hierarchy in `text/diagrams/FrontendPlayback.png`, and the multi-tab sync sequence in `text/diagrams/PlaybackSync.png`.

Playback is server-authoritative. The Backend keeps one `PlaybackState` object per User (covering all User connections) in Redis. Every WS action from any client updates that state, and the server then broadcasts the updated state to every connected User tab.

An earlier design let each tab keep its own state and used the server only to pass updates to the other tabs. That design was dropped because the server could not order the events. Suppose tab A sends a seek to 30 s and tab B sends a seek to 90 s 50 ms later. Network jitter could swap the order in which the two broadcasts reach a third tab, so its position would end at 30 s even though 90 s was the newer action. The server had no way to pick a winner. The server-authoritative model fixes this because every state change goes through one ordered point and gets a fresh version number, described in the next subsection.

6.9.1 Shared state and version counter

`PlaybackState` has two fields. The first is `ActivePlayback`, the part of the state which has the actual Playback-related values. It holds the current Song UUID, the play instance UUID, an is-playing flag, and the position pair (Song position and the server time at which it was read). `ActivePlayback` is a separate object because it only exists while a Song is loaded (when nothing is playing, the field is `None`). The second field is a monotonically increasing version number kept on `PlaybackState` itself, so it survives across pauses and Song changes.

The version increments on every “real” state change (seek, pause, etc., not clock sync). Every event sent to clients includes the current version. A tab keeps the version of its last applied event and drops any broadcast whose version is older than that, which protects against out-of-order arrivals like the one described above. The Backend itself does not use the version to drop incoming actions. Instead, every mutating action from a tab includes the Song UUID the tab thinks is current, and the Backend drops the action if that UUID does not match the one in `ActivePlayback`. A mismatch means the tab is acting on a Song the User has since moved past.

The play instance UUID is regenerated when a new Song becomes the active Playback item. It disambiguates Queue entries that point at the same Song. Queue entries are `CollectionSongs`, and the User can put the same `CollectionSong` into the Queue more than once in a row. Without the UUID, resuming after a skip to the second occurrence would inherit the position saved for the first.

6.9.2 Position estimation on passive tabs

Exactly one Device is the active Playback Device at any time. Only the active tab loads the current Song into its HTML audio element. If every tab loaded the Song, the User would hear several streams at once.

The audio element itself stays mounted on every tab, even when empty. The Web Audio API pipeline (covered later in this section) binds to this element. Building the pipeline needs a User gesture, so the Frontend builds it once on the first gesture (see the autoplay subsection below). Keeping the element mounted on every tab lets the same pipeline survive tab switches.

The active tab reads the current Song position directly from its audio element. Passive tabs do not have a Song loaded, but they still need to show the position slider. They therefore estimate the position from the server’s last known value.

Only the active tab reports the position to the server. It reads the `currentTime` attribute of its audio element and sends that value. The server stores the pair `last_known_song_pos_ms` (the reported position) and `last_known_at_server_ms` (the server clock when it arrived). Any client estimates the current position as `last_known_song_ -`

`pos_ms` plus the time that has passed since `last_known_at_server_ms`. Clients do not have direct access to the server clock. Each broadcast includes the server's current timestamp, and the client compares it with its own local clock at receive time to learn how far the two clocks are apart. It then adds that gap to its local clock whenever it needs to estimate what the server clock reads now. The active tab updates the pair when its audio position diverges from its own estimate by more than 500 ms, and on a 20 s heartbeat otherwise. The heartbeat is needed because the drift check itself depends on the active tab's estimate of the server clock, which gets less accurate without fresh broadcasts.

6.9.3 Device activation

The active Device id is tracked on the server alongside the Device list, in a separate Redis key from `PlaybackState`. The Backend never fills it automatically. A User can have several tabs and Devices open at once, and the Backend cannot tell which one the User currently uses. Another reason to avoid an auto-pick is the browser autoplay policy: the User click that sets the active Device also counts as the browser gesture needed to start audio on that tab (see the next subsection).

When no Device is active, every tab is passive. The first play action from the User sets the active Device id. The Frontend on the tab that issued the play checks whether an active Device exists, and if not it sends `device.set_active` for its own Device id before the `playback.activate`. Skipping to the next or previous Song goes through the same play action path, so it sets the active Device the same way. The seek action does not go through the play action path on the Frontend, because the position slider is its own UI component. The Backend therefore picks the Device that sent the seek when no Device is active yet, so the next play does not need a separate `device.set_active`.

A change of active Device is a `device.set_active` WS action (initiated by the Device dropdown) followed by a broadcast of the new Device list. The Backend pauses Playback on the transition. Without the pause, the newly active tab would try to resume audio without a User gesture on that tab, which the browser would block (again, see the next subsection on autoplay).

6.9.4 Browser autoplay handling

Browser autoplay policies add two constraints. First, browsers only let the page create or resume an `AudioContext` (the Web Audio API starting point used by the pipeline in the next subsection) while the browser is still handling a User click or key press. The Frontend listens for the first `pointerdown` or `keydown` anywhere on the page and builds the pipeline inside that handler, so the first User action counts as the gesture. Second, `audio.play()` itself is gesture-gated. If the User had another tab playing, then closed it and resumes from a tab they have not interacted with yet, `audio.play()` rejects with `NotAllowedError`. The Frontend catches this and retries with the audio element muted, which browsers always allow, and the UI shows an Unmute control. The Unmute click is the gesture that lets the Frontend resume the `AudioContext` and switch the audio element back to unmuted.

6.9.5 Frontend playback control

The Frontend Player is a class, `AudioPlayerController`, not a React hook. An earlier version of the Frontend used a hook (named `usePlayer`). In that version, each Websocket handler held a “frozen copy” of the React Playback state from the moment it was attached, and was never re-attached, so later events kept reading the old values. The class form removes this problem because it owns the Playback state directly as a field. React components subscribe to the class through `useSyncExternalStore` and re-render on changes, while the WS handlers read the same field directly and always see the current value.

The Playback needs per-Song loudness normalization, and the HTML audio element does not offer it. The Frontend therefore routes audio through a Web Audio API graph between the audio element and the speakers. The graph is owned by an `AudioContext`, the starting point for any Web Audio API work. It has four stages. The first gain stage applies the normalization, using the stored LUFS measurement of the Song. The second is the User’s volume slider. The third adds short volume fades around play, pause, and seek, because abrupt cuts make audible clicks. A compressor at the end catches the peaks, because a loudness boost on a quiet Song can push the signal past the clipping point.

With Playback covered, the next section turns to Publications: Posts, Collection Comments, Chat messages, and Replies.

6.10 Publications

This section is about Posts, Collection Comments, Chat messages, and Replies. The Backend code is under `src/backend/musikk/social`, the Frontend code under `src/frontend/musikk/src/features/publications`. The class diagram is in `text/diagrams/SocialClassDiagram.png`, the Frontend component hierarchy in `text/diagrams/FrontendPublications.png`, and the Chat connection lifecycle in `text/diagrams/ChatLifecycle.png`.

All four kinds use the same Django model, `Publication`. The model stores the author, the text, an optional parent `Publication` for Replies, and a `created_for` field that points to the object the `Publication` is written for. This object is a `User` for Posts, a `Collection` for Collection Comments, and a `Chat` for Chat messages. An optional attachment field can point to a `Song` or a `Collection`. The shared model was chosen because every kind has the same basic shape. A `User` writes some text. The text can be a Reply to another `Publication`, and it can have a `Song` or a `Collection` attached. This shared model has some downsides that are discussed later.

At first, the API was built the same way. One endpoint took the `created_for` type and id, and then chose the right logic inside. This became hard once the rules for each kind started to be different. Collection Comments are only allowed on public, non-draft Albums or Playlists. Posts are only allowed on the User's own Feed. Chat messages are only allowed between Chat members. This needs two extra models, `Chat` and `ChatMember`, which do not fit into the shared row. The side effects are also different. A new Collection Comment sends an event on the `collection_comments.<uuid>` topic. A new Chat message sends events on the `chat.<uuid>` topic and on the user group of every other Chat member. A new Post sends nothing, because the Feed list is reloaded by HTTP polling. Each kind also has its own Notification rules. A Reply creates a Reply Notification for the parent author, and a Chat message creates a Chat message Notification for every other Chat member. With one endpoint, every request had to

check `created_for_type` right at the start to choose the permissions, the event, and the Notification.

The API was therefore split into one endpoint per kind, `/feed/<user_uuid>/posts`, `/collections/<uuid>/comments`, and `/users/<user_uuid>/chats/<chat_uuid>/messages`. Each view keeps the rules for its kind in one place, so this branching is no longer needed. A shared mixin, `PublicationsListCreateMixin`, still holds the common create and list code. The per-kind views only override the permission checks, the `created_for` lookup, and the post-create hook. Replies work the same way for all four kinds, because a Reply has the same shape regardless of its root Publication. They are created through the same endpoint as the root, with the parent field set. They are read through one shared endpoint, `/publications/<uuid>/children`. The database keeps one table for all kinds, and the API uses the names that the Frontend already expects.

6.11 Search

This section covers full-text Search across Songs, Albums, Playlists, Users, and Artists. The Backend code is under `src/backend/musikk/recommendations`, the Frontend code under `src/frontend/musikk/src/features/search`.

6.12 Notifications

This section shows how Notifications are modelled, emitted, and delivered. The Backend code is under `src/backend/musikk/notifications`, the Frontend code under `src/frontend/musikk/src/features/notifications`. The class diagram is in `text/diagrams/NotificationsClassDiagram.png` and the Frontend component hierarchy in `text/diagrams/FrontendNotifications.png`.

There are three Notification kinds. The first is a Reply to a Publication the User authored. The second is a new Follower. The third is a new Chat message. The first design used one shared Notification base with three concrete subclasses through Django's multi-table inheritance. Each row held an `is_read` boolean. This worked well at the start. Every kind had the same two operations, list and mark read,

and one query covered all of them. However, the kinds soon gained more fields of their own, and the shared base no longer fit well. A Reply points to two Publications, a Follower to two Users, and a Chat message to a Chat together with the message and the receiver. The columns almost do not overlap. As a result, a shared row was mostly nulls, and every read needed an extra join into one of the concrete tables. In contrast to Publications, which still use one shared model with mostly-null rows, Notifications have been split into three separate models.

Each kind is its own `BaseModel` subclass with its own `Serializer` and custom `Manager`. In comparison to other parts of the application, managers here are used to emit a `notifications.changed` WS event instead of the API views.

Read state was also made simpler. The per-row `is_read` boolean was replaced by one `NotificationProfile.read_at` timestamp per User. The profile is created on User registration through a `post_save` signal handler. The `Serializer` computes `is_read` for each row by comparing `date_added` against that timestamp. As a result, “mark all as read” becomes one field update.

What still stays shared is the REST API seen by the Frontend. One `GET /notifications` endpoint returns a response with three buckets. A `PATCH` on the same path updates `read_at`. A `DELETE /notifications/<uuid>` checks the three tables in turn to find the matching row. This shape still suits the Frontend, which fetches all kinds together for the badge and the list view. However, the differences between kinds are no longer hidden behind a common base.

6.13 Security

6.14 Deployment

This section covers the production deployment setup. The Docker Compose file is at `docker-compose-prod.yml`. The Backend and Frontend Dockerfiles are at `src/backend/Dockerfile` and `src/frontend/Dockerfile`, and the Caddy configuration at `src/frontend/Caddyfile.prod`.

The Backend, the Postgres database, the Redis instance, and the Celery worker run as four containers under Docker Compose on a single Hetzner VPS. The Frontend is built into static files at image-build time and served by the same Backend container. Caddy [87] is the reverse proxy in front of them. Mail is created by the Mailgun service via `django-anymail`, and the domain is managed at PorkBun. FFmpeg is built from source inside the container with `libfdk_aac` enabled, since the prebuilt distributions ship without it and AAC encoding quality on the bundled native encoder is visibly worse. The resulting image is not published in a public registry, since the FFmpeg with `libfdk_aac` binary cannot be redistributed due the licensing issues [88].

A Survey Questionnaire

Music Consumption Study

This short form is designed to analyze the people's behaviour related to music - ways of discovering new audio pieces, the role of music in social interactions, preferred ways of consuming audio information etc.

1. How old are you?

Mark only one oval.

- ☐ 0-18 years old
☐ 18-30 years old
☐ 30-60 years old
☐ 60+ years old

2. How do you mostly consume audio media?

Tick all that apply.

- ☐ Streaming platforms - Spotify, Apple Music, Yandex Music and others.
☐ Vinyl, CD discs and other physical media.
☐ Downloaded files.
☐ Other: _____

3. How often do you listen to music?

Tick all that apply.

- ☐ Everyday.
☐ Often, but not everyday - e.g. 3-5 days a week.
☐ Once or twice a week at most.
☐ Only on special occasions.
☐ Other: _____

A. SURVEY QUESTIONNAIRE

Music Consumption Study

https://docs.google.com/forms/u/5/d/1vhhAu_SfuHV4xy6JoDaRsM5...

4. Roughly - how much songs do you listen to in a month?

Tick all that apply.

- ☐ 1000+
☐ 500-1000
☐ 100-500
☐ 50-100
☐ 10-50
☐ Less than 10
☐ Other: _____

5. Would you say that you are consistent with the music you listen to? I.e. do you mostly listen to the same artists/songs/genres?

Mark only one oval.

- ☐ Yes
☐ Mostly
☐ Rather no than yes
☐ No

6. How often do you listen to something new?

Mark only one oval.

- ☐ Very rarely, once a couple of months.
☐ Rarely, maybe a handful times a month.
☐ Pretty often, every week or so.
☐ Often, a couple of times a week.
☐ Every or almost every day.

A. SURVEY QUESTIONNAIRE

Music Consumption Study

https://docs.google.com/forms/u/5/d/1vhhAu_SfuHV4xy6JoDaRsM5...

7. How do you find out about new music?

Tick all that apply.

- ☐ At random - in cafes, shops and other venues.
- ☐ From people I know.
- ☐ Non-specialized online platforms - Instagram, TikTok, Youtube(videos), etc.
- ☐ Specialized online platforms - Rate Your Music, Pitchfork, Wire, etc.
- ☐ Games, movies, other forms of digital content.
- ☐ Buying new physical releases(vinyl, cds ...)
- ☐ Recommendations on streaming platforms
- ☐ Other: _____

8. Would you say that you know a lot of people with similar music taste?

Mark only one oval.

- ☐ Yes
- ☐ No

9. Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?

Mark only one oval.

- ☐ Yes
- ☐ No, I already have enough people around me with whom I can share the music I like
- ☐ No, I do not feel the need in that
- ☐ Other: _____

A. SURVEY QUESTIONNAIRE

Music Consumption Study

https://docs.google.com/forms/u/5/d/1vhhAu_SfuHV4xy6JoDaRsM5...

10. Are there any situations, when you listen to music with someone else?

Tick all that apply.

- ☐ At parties; when I'm visiting someone; in a car.
- ☐ Online via Spotify Jam, Discord bots and other similar instruments.
- ☐ Concerts, clubs, street performances.
- ☐ At work.
- ☐ Other: _____

11. How often do you listen to music with someone else?

Mark only one oval.

- ☐ Very rarely, maybe a couple times a year.
- ☐ Rarely, every one or two months.
- ☐ Often, a couple of times a month.
- ☐ Very often, every week or so.
- ☐ Almost every day.
- ☐ Other: _____

12. How often do you/your friends share/discuss music?

Mark only one oval.

- ☐ Very rarely, maybe a couple times a year.
- ☐ Rarely, every one or two months.
- ☐ Often, a couple of times a month.
- ☐ Very often, every week or so.
- ☐ Almost every day.
- ☐ Other: _____

A. SURVEY QUESTIONNAIRE

Music Consumption Study

https://docs.google.com/forms/u/5/d/1vhhAu_SfuHV4xy6JoDaRsM5...

13. How does that happen?

Tick all that apply.

- ☐ Online, sending links from streaming platforms via messengers.
- ☐ Offline, in a car, at someone's place etc.
- ☐ Via sharing the physical media(e.g. giving a vinyl to a friend for a listen)
- ☐ Other: _____

14. Question for users of streaming platforms.

What platform do you use?

Tick all that apply.

- ☐ Spotify
- ☐ Apple Music
- ☐ Yandex Music
- ☐ VK Music
- ☐ SoundCloud
- ☐ Bandcamp
- ☐ Youtube Music
- ☐ Other: _____

15. Question for users of streaming platforms.

Would you benefit, if current streaming platforms added extra social features(i.e. would you use them)? For example, chat discussions for playlists and songs, feeds with music added by the people you follow, forums, direct messages, etc.

Mark only one oval.

- ☐ I would use these features often.
- ☐ I would use these features rarely.
- ☐ I wouldn't use these features.

A. SURVEY QUESTIONNAIRE

Music Consumption Study

https://docs.google.com/forms/u/5/d/1vhhAu_SfuHV4xy6JoDaRsM5...

16. Question for users of streaming platforms.

Are there any other features you would like to see?

This content is neither created nor endorsed by Google.

Google Forms

A. SURVEY QUESTIONNAIRE

Music Consumption Study

https://docs.google.com/forms/u/5/d/1vhhAu_SfuHV4xy6JoDaRsM5...

B Audio Processing Pipeline Details

These notes expand on the *Audio Processing and Streaming* section of **Chapter 6**. The main chapter names every stage briefly and lists the chosen parameters. The sections below give the deeper rationale for each choice and a per-flag breakdown of every CLI invocation.

B.1 Streaming Protocols

At first, only DASH was considered, because it works better in varying network conditions [89] and is not limited to a small set of audio codecs. [23]

However, DASH support in audio players is typically implemented via browser Media Source Extensions [90], which iOS does not support. [91] Since iOS accounts for roughly 15% of all operating systems worldwide [92], HLS support had to be added as well.

Both protocols are therefore used: DASH on devices that support MSE, and HLS on iOS.

B.2 Packaging

Because the platform serves both DASH and HLS, it is useful to generate only one set of chunks and reuse it for both streaming methods. For this reason, the packaged output follows Common Media Application Format (CMAF), which defines a common way to store chunks as fragmented MP4 (fMP4) containers. [93] This is usually not stated explicitly by packager utilities, but can be observed by running, for example, the `ffprobe` CLI utility on a packaged segment.

The resulting output contains one initialization segment (`*_init.mp4`), which stores audio metadata and decoder setup, and a sequence of media segments (`*.m4s`), which store the audio data in short time intervals. In addition, this approach is required for streaming FLAC encoded (lossless) audio via HLS. [94]

B.3 Processing Tools

Three CLI tools are used in the processing pipeline: FFmpeg for Transcoding and audio analysis, FFprobe for reading audio metadata, and Shaka Packager for the Packaging step.

FFmpeg [71] is an open-source CLI tool. It supports all of the selected codecs, can be run programmatically as a script, and converts audio at high speed because it is written in C and is optimized to use multiple threads. The pipeline uses it both as the transcoder and as the host for the filter graph applied during Audio Preprocessing.

FFprobe ships with the FFmpeg project and reads metadata from audio files. The Audio Preprocessing pipeline uses it to read the source codec, sample rate, channel count, bit depth, duration, and bitrate of every upload before transcoding.

Even though FFmpeg is able to do the Packaging step as well, a separate packager tool was chosen. This is due to the problems that FFmpeg had with generating manifests which are compatible with Shaka Player [73], used on the Frontend for Playback. As a consequence, Shaka Packager was selected for the packaging step. [72] It supports both DASH and HLS, can generate CMAF-style fMP4 segments, and can produce both manifests in one run. In addition, it integrates well with Shaka Player.

B.4 Transcoding

B.4.1 Why transcoding is needed

Usually, when high-quality audio is recorded, it is initially saved in raw audio formats, such as WAV, which results in big file sizes that are impractical for Streaming. For instance, the resulting WAV file for a 3-minute stereo audio recording with a bit depth of 24 bits and a sample rate of 44.1 kHz is approximately 50 MB.

To reduce the size, the audio is usually processed using a special program called a *codec*, which compresses and stores audio in a more efficient format.

Lossless codecs, such as FLAC [95], keep all audio information, so the audio sounds exactly the same as the original raw data. In contrast, lossy codecs (MP3 [96], AAC [97], OPUS [98], etc.) remove

some information to reduce file size, which can slightly reduce quality, especially at low bitrates.

In this implementation, the lossless FLAC codec and lossy AAC/HE-AAC codecs were selected due to their good performance in web-based streaming scenarios. [78, 79, 80]

Other codecs that promise better sound quality at smaller file sizes, such as Opus, were also considered. However, only *-AAC and FLAC/ALAC codecs are supported for HLS streaming. [77]

B.4.2 AAC encoder choice

FFmpeg ships several AAC encoder implementations, only one of which is used here:

- `aac_at` is the best available codec in terms of quality. [99, 100, 101] However, it was excluded due to its licensing policy, which states that this codec could be used only on Apple-branded devices. [102, 88]
- `aac` is FFmpeg's own AAC implementation. It was also rejected, since the quality it provides is sub-par (even stated by the official documentation), behind the other two codecs. [99, 100, 101, 88]
- `libfdk_aac` is the second best codec in terms of quality and the codec that was eventually chosen. It also has licensing issues: any binaries, Docker images, and other distribution methods which use it cannot be made public or freely shipped to other devices. However, it is available for use free of charge in closed-source projects. [103, 104, 88]

For FLAC, FFmpeg provides a single `flac` encoder, which is used directly.

B.4.3 Bitrate strategy: CBR versus VBR

Codecs usually provide several parameters that affect the output, such as sample rate, the number of channels, and bitrate. Bitrate describes how much audio data is stored or transmitted per second and is typically expressed in kilobits per second (kbps). It is commonly used

because it provides a direct way to control the trade-off between audio quality and file size.

Before selecting the encoding configuration, it is also important to distinguish between two encoding strategies: Constant Bitrate (CBR) [105] and Variable Bitrate (VBR). [106]

CBR keeps the bitrate fixed across the entire audio file, so each representation has a known, stable bandwidth. VBR adjusts the bitrate depending on content complexity, which gives better quality at a smaller size compared to CBR, but the per-segment size varies within a single representation. [107]

The platform serves several tiers in parallel and lets the Frontend's Adaptive Bitrate (ABR) logic pick one based on the measured network throughput. A stable per-tier bandwidth is the input ABR needs, so a multi-CBR setup was selected. The same effect could in principle be reached with constrained VBR tiers, but the simplicity of a fixed per-tier bitrate was preferred for the project scope.

B.4.4 Generation loss

Transcoding a lossy source into AAC causes generation loss: every lossy pass discards some audio, so re-encoding what was already lossy compounds the damage. The pipeline still performs the conversion because DASH and HLS need every representation in the same codec. With FLAC and AAC chosen as the output codecs, lossy uploads have to be re-encoded into AAC anyway.

B.5 Audio Preprocessing

B.5.1 Input validation limits

The upload passes through a chain of validation checks. The file-size and MIME checks run on the raw bytes. The remaining constraints check values read from the file by FFprobe. Raw-audio storage scales linearly with duration, sample rate, channel count, and bit depth, so the constraints together bound the worst-case storage and Transcoding time of a single upload:

- **MIME type.** The first 2048 bytes of the file are inspected with `python-magic` [83], and the type must start with `audio/`. The

subtype must also belong to an allow-list of subtypes that map to the supported codecs, defined in the `ALLOWED_FILE_TYPES` variable in `streaming/audio/config.py`. This step detects malformed or non-audio uploads before any audio tool is invoked.

- **Codec allow-list.** A container can contain different codecs (for example, an `.mp4` file can hold AAC or ALAC), so the codec name FFprobe reports must also belong to `ALLOWED_CODECS` in `streaming/audio/config.py`. The list only covers popular codecs, since FFmpeg's decoders for rare codecs are less tested and can produce a Transcoded file with wrong duration or corrupted samples.
- **File size: 2 GB.** Roughly a 2-hour 24-bit, 48 kHz stereo WAV. An arbitrary number, but real music files usually stay well below this, so the cap mostly guards against accidental or malicious oversize uploads.
- **Duration: up to 7200 s (2 h).** Covers DJ sets and long mixes while keeping the upload and Transcoding time reasonable.
- **Sample rate: 1 Hz to 384 kHz.** The upper bound matches the highest rate in the supported sample rate families, described in the Sample rate normalization step below.
- **Channel count: 1 to 8.** Covers mono through 7.1 surround, which is the practical upper bound for source audio.

B.5.2 FFprobe invocation

```
ffprobe -v error -select_streams a \
  -show_entries stream=codec_name,sample_rate,channels,bits |
  ↪ _per_raw_sample,duration,bit_rate \
  -show_entries format=duration,bit_rate \
  -of json input.wav
```

The flags are:

- `-v error` suppresses informational and warning output, keeping only actual errors.

- `-select_streams` restricts probing to audio streams.
- `-show_entries stream=...` returns only the per-stream fields the wrapper consumes.
- `-show_entries format=...` adds a fallback for `duration` and `bit_rate`, since some containers (notably WAV) only report them at the format level.
- `-of json` produces structured output that the wrapper parses.

B.5.3 Tier selection

Tier selection drops output tiers that cannot help the listener. A lossy source skips the FLAC tier, since re-encoding lossy audio as lossless cannot recover the information the source codec already discarded. It would only add storage. An AAC tier whose target bitrate sits above the source bitrate is dropped for the same reason: lossy re-encoding cannot add detail above what the source already carries.

B.5.4 Channel normalization

The normalization step reduces every track to mono or stereo. Music releases are mostly mastered in stereo, and the chosen Streaming Protocols and the loudness measurement step below are also defined for mono or stereo input. Surround formats such as Dolby Atmos need a different container, extra information in the manifest, and per-platform decoder support, none of which fit the project scope.

Mono inputs stay mono, since upmixing to stereo would not add information, only file size. Stereo inputs stay stereo. Sources with three or more channels are downmixed to stereo by inserting `aresample=out_chlayout=stereo` into the FFmpeg filter chain.

B.5.5 Sample rate normalization

Human hearing tops out around 20 kHz. The Nyquist-Shannon theorem says a sample rate must be at least twice the highest frequency in the signal to represent it without aliasing. A 44.1 kHz rate therefore covers the audible band with a small margin, and source rates above that carry no perceptual gain, only larger files.

Music masters use one of two standard rates, 44.1 kHz or 48 kHz, or an integer multiple of one of them. The step groups each source into its family and resamples down to the family base. An integer-ratio resample is cheap to compute and stays close to the original signal. Files below 44.1 kHz are left as-is, since upsampling adds no information. Non-family rates default to 44.1 kHz. Resampling is done by inserting `aresample=<rate>` into the FFmpeg filter chain.

B.5.6 Bit depth conversion

All audio is converted to signed 16-bit samples via `aformat=sample_fmts=s16`. The conversion is unconditional because `libfdk_aac`, the encoder used for the AAC tiers, does not accept higher-precision sample formats. FLAC accepts higher precision, but feeding the same filter chain to every tier keeps the Transcoding command uniform.

B.5.7 Loudness theory

Loudness normalization keeps perceived volume steady across tracks. Human hearing is not flat: equal-loudness contours described by ISO 226 show that perceived loudness depends on both amplitude and frequency, so two files with the same digital peak can sound very different. Without normalization, a playlist that mixes tracks from different masters would have the listener constantly reaching for the volume knob.

ITU-R BS.1770 is the standard algorithm for measuring loudness. It produces an integrated value in Loudness Units relative to Full Scale (LUFS), using a frequency weighting (K-weighting) shaped to approximate human hearing and a channel weighting model. EBU R128 wraps the algorithm into a normalization recommendation, with a broadcast target of -23 LUFS. Streaming services use higher targets because listening conditions differ from broadcast, for example Spotify at -14 LUFS and Apple at -16 LUFS.

Two other units appear here. Decibels Full Scale (dBFS) measure sample amplitude relative to the maximum representable value, so 0 dBFS is the largest sample and recorded peaks are typically negative. True peak in dBFS (dBTP) measures the peak of the reconstructed analog waveform after digital-to-analog conversion, not the peak of

the stored samples. The distinction matters: a sequence of samples that all sit below 0 dBFS can still reconstruct to over 0 dBFS, producing inter-sample peaks that clip on Playback. True-peak measurement uses an oversampled signal to catch this.

B.5.8 Why loudness is measured after Transcoding

FFmpeg's `ebur128` filter with `peak=true` produces both an integrated LUFS value and a true peak in dBTP. The measurement runs after Transcoding, on the FLAC output if it was produced, otherwise on the first AAC output. FLAC is preferred when available because it preserves the normalized signal bit-perfectly. The measured values therefore describe the normalization output without being nudged by the lossy AAC encoder.

The measurement happens after Transcoding rather than on the source file for two reasons. First, the listener hears the post-processing audio. Channel downmix changes overall loudness, since each channel contributes to the perceived sum. Sample-rate conversion can also shift true peaks slightly. The stored values therefore have to describe what actually plays. Second, the `ebur128` summary reports the peak as a per-channel list on multichannel inputs. For example, a six-channel source produces a summary line such as `Peak: -1.1 -2.3 -1.5 -2.1 -3.0 -1.8 dBFS`, which is ambiguous as a single per-track value. After downmix to stereo the list has at most two entries, and the wrapper records the maximum of the two.

B.6 Pipeline invocations

B.6.1 FFmpeg transcoding flags

The FFmpeg invocation for the AAC 160 kbps tier on a stereo 44.1 kHz source is:

```
ffmpeg -i input.wav \
  -vn -sn -dn \
  -af "aresample=out_chlayout=stereo,aresample=44100,aformat=
    ↪ t=sample_fmts=s16" \
  -c:a libfdk_aac \
  -b:a 160k \
```

```
-movflags +faststart \  
libfdk_aac-160-<uuid>.mp4
```

The flags are:

- `-i input.wav` selects the input media file.
- `-vn -sn -dn` strip any video, subtitle, and data streams so only the audio stream is processed.
- `-af "..."` applies the audio filter graph for normalization (channel downmix, sample-rate normalization, bit-depth conversion).
- `-c:a libfdk_aac` selects the audio encoder. The other encoders used by the pipeline are `flac` for the lossless tier and `libfdk_aac` with the additional `-profile aac_he_v2` flag for the HE-AACv2 24 kbps tier.
- `-b:a 160k` sets the target audio bitrate. This flag is omitted for the FLAC tier, since it is lossless.
- `-movflags +faststart` moves the MP4 metadata (the `moov` atom [108]) to the start of the file, so Playback can begin before the whole file is downloaded.
- The trailing positional argument is the output file path. The wrapper uses the pattern `<encoder>-<bitrate>-<uuid>.mp4` for the lossy tiers and `flac-static-<uuid>.mp4` for the lossless tier.

B.6.2 Shaka Packager flags

The Shaka Packager invocation for two of the AAC tiers is:

```
packager \  
'input=/tmp/in/libfdk_aac-96-<uuid>.mp4,stream=audio,init_s|  
  → egment=/tmp/out/audio_0_init.mp4,segment_template=/|  
  → tmp/out/audio_0_$.m4s,playlist_name=audio_0.|  
  → m3u8' \  

```

```
'input=/tmp/in/libfdk_aac-160-<uuid>.mp4,stream=audio,init_」
  → segment=/tmp/out/audio_1_init.mp4,segment_template=」
  → /tmp/out/audio_1_${Number$.m4s,playlist_name=audio_1」
  → .m3u8' \
--segment_duration 3 \
--generate_static_live_mpd \
--mpd_output=/tmp/out/manifest.mpd \
--hls_master_playlist_output=/tmp/out/master.m3u8
```

The Backend builds this command as a Python list and invokes it via subprocess, without a shell, so the `${Number$}` token is passed to the packager literally. The shell rendering above wraps each per-input argument in single quotes to prevent bash from expanding it.

The flags are:

- packager is the Shaka Packager binary.
- Each `input=...` argument defines one representation. The keys are:
 - `input=<path>` is the local path to the transcoded audio file.
 - `stream=audio` selects only the audio stream from that file.
 - `init_segment=<path>` sets the output path for the CMAF/fMP4 initialization segment (`*_init.mp4`).
 - `segment_template=<path>` sets the template for media segment file names. `${Number$}` is a placeholder that the packager substitutes with the segment index.
 - `playlist_name=<name>` sets the file name of the per-representation HLS variant playlist, for example `audio_0.m3u8`.
- `-segment_duration 3` sets the target segment length in seconds. The value of 3 s was chosen as a balance between bitrate-switching responsiveness and HTTP request overhead.
- `-generate_static_live_mpd` switches the DASH manifest type from dynamic (the default) to static, which is what audio-on-demand clients expect and what enables seeking through the track.

- `-mpd_output=<path>` sets the output path for the DASH manifest (`manifest.mpd`).
- `-hls_master_playlist_output=<path>` sets the output path for the HLS master playlist (`master.m3u8`).

B.6.3 ebur128 loudness measurement flags

The FFmpeg invocation for the loudness pass, run on the FLAC output if it was produced or otherwise on the first AAC output, is:

```
ffmpeg -i flac-static-<uuid>.mp4 \
  -filter:a ebur128=peak=true \
  -f null -
```

The flags are:

- `-i <path>` selects the transcoded audio file the wrapper picked for measurement.
- `-filter:a ebur128=peak=true` runs the ebur128 audio filter. `peak=true` enables true-peak measurement (dBTP) alongside the integrated LUFS value, so the wrapper gets both numbers from a single pass.
- `-f null -` discards the audio output, since only the summary the filter prints on stderr is needed. The trailing `-` tells FFmpeg to write the (discarded) output to stdout.

The wrapper parses the printed summary, extracts the I: (LUFS) and Peak: (dBTP) values, and stores them on the Song row.

C Diagram Catalog

The diagrams and charts below illustrate the structure and behavior of the application, along with the survey results, and are referenced from **Chapter 2** and **Chapter 4**. The source PlantUML files live under `specification/`.

C.1 Use Case Diagrams

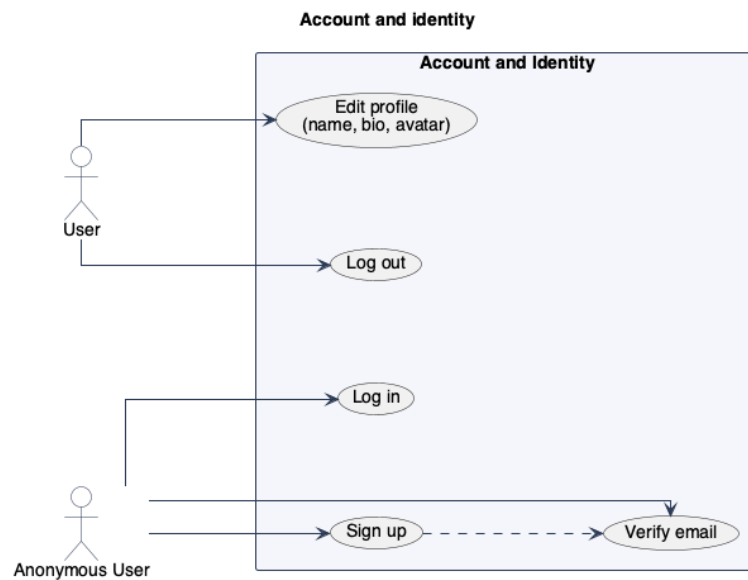


Figure C.1: Use cases: Account and Identity

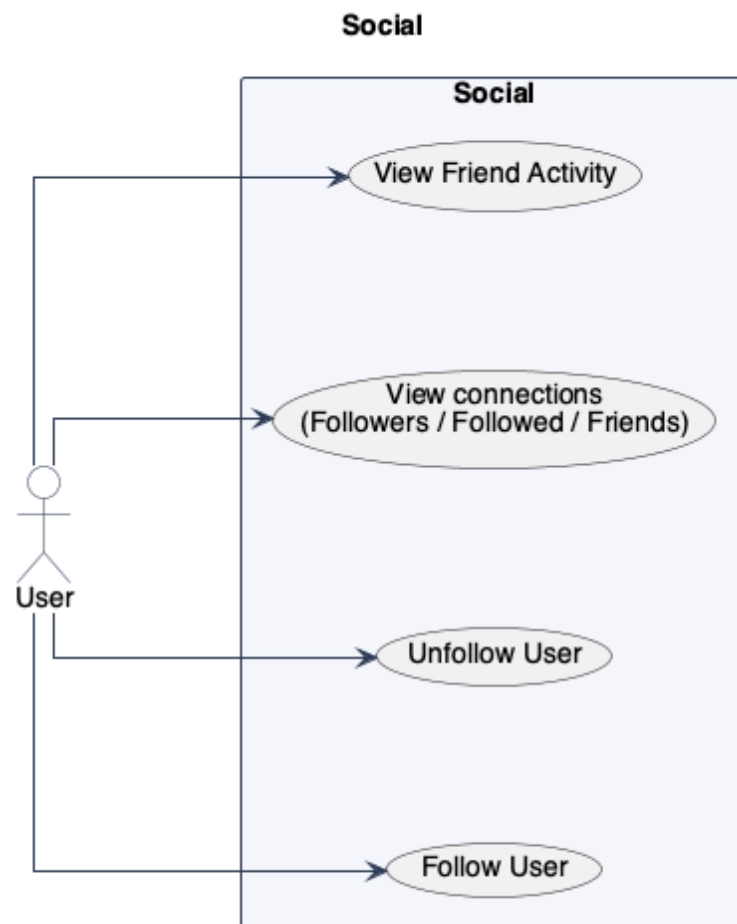
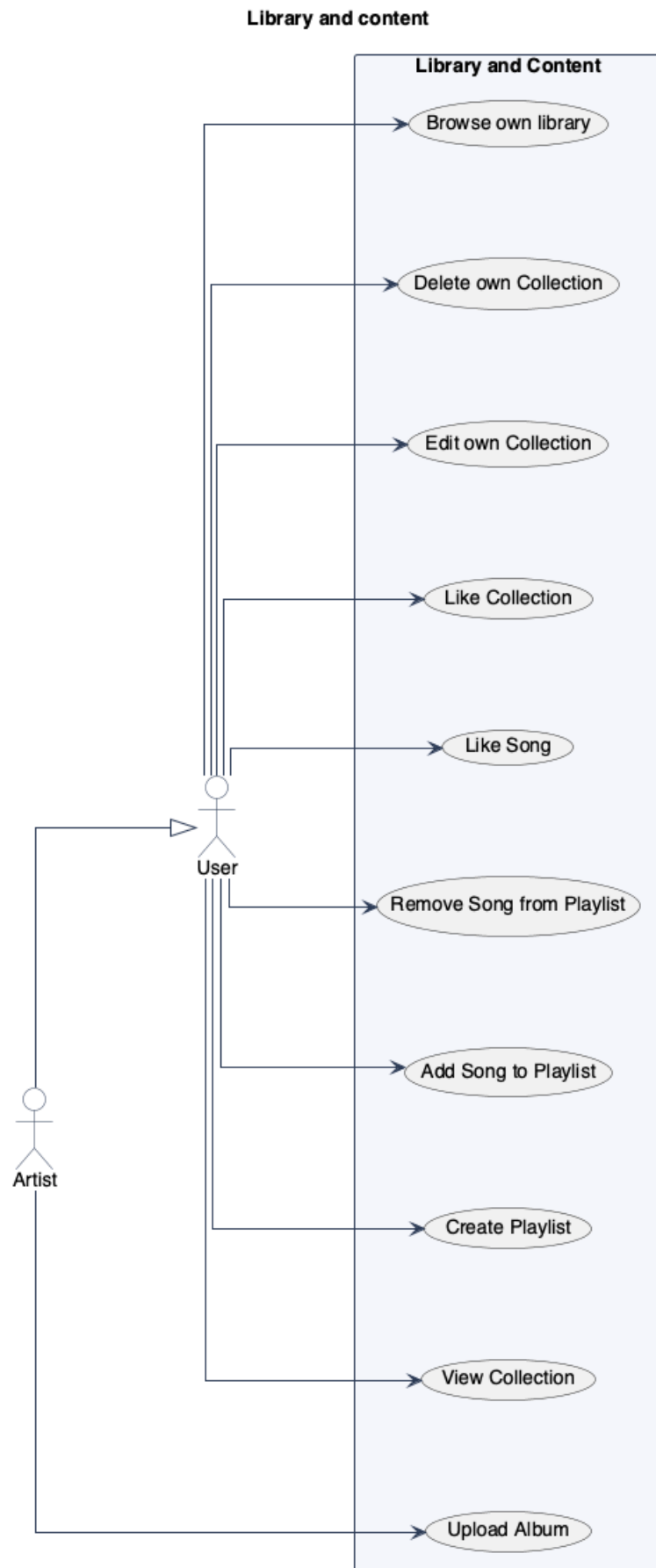


Figure C.2: Use cases: Social

**Figure C.3:** Use cases: Library and Content

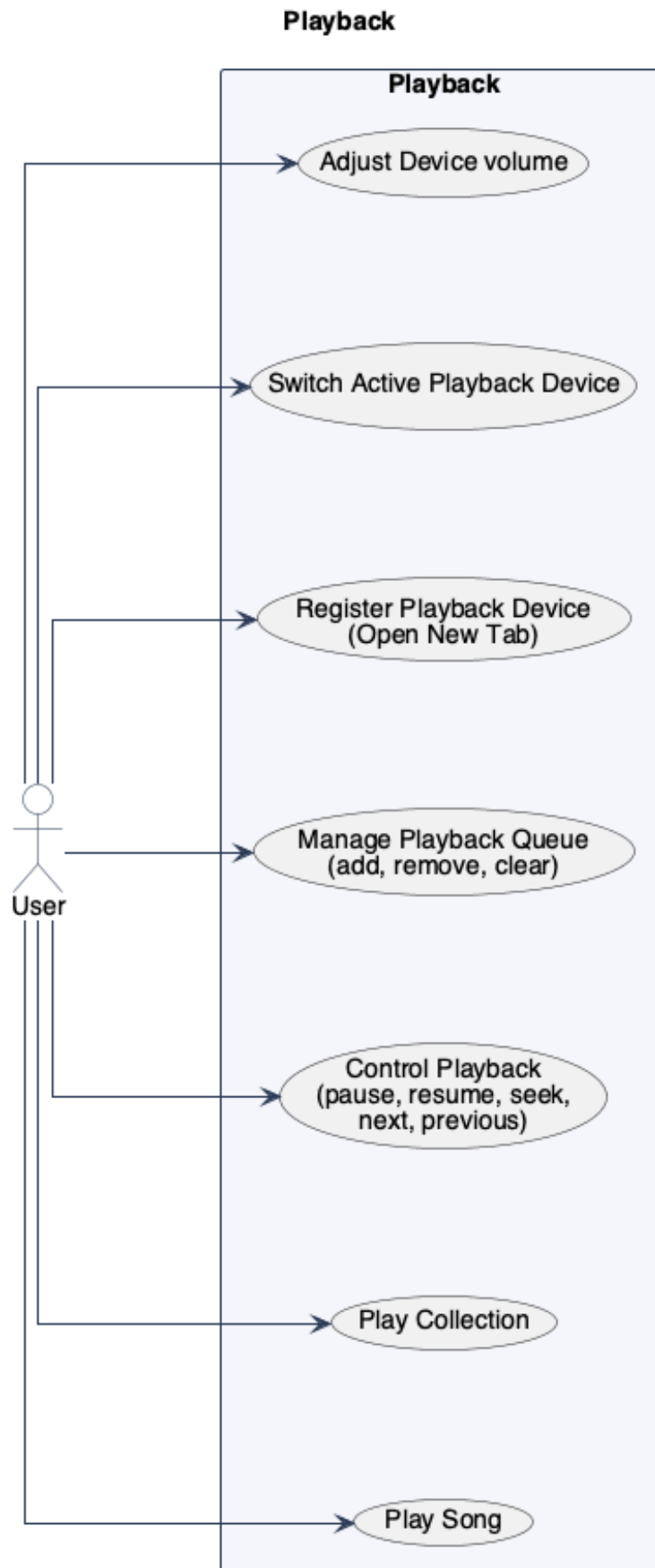


Figure C.4: Use cases: Playback

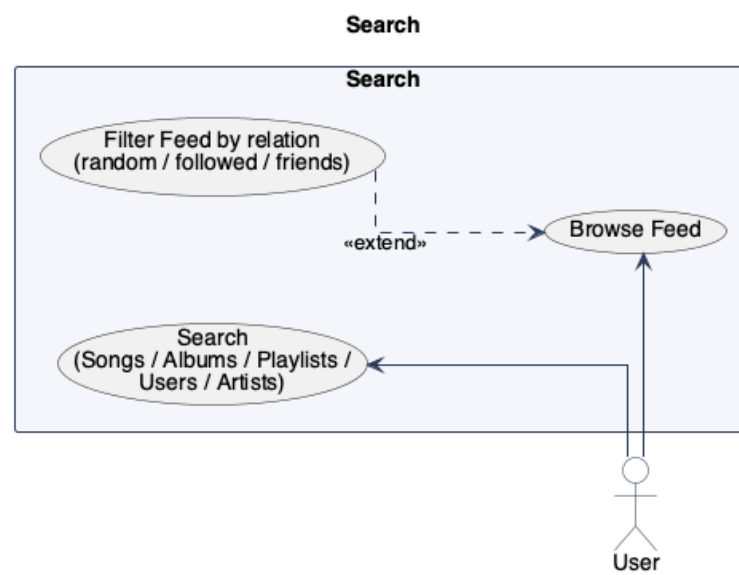
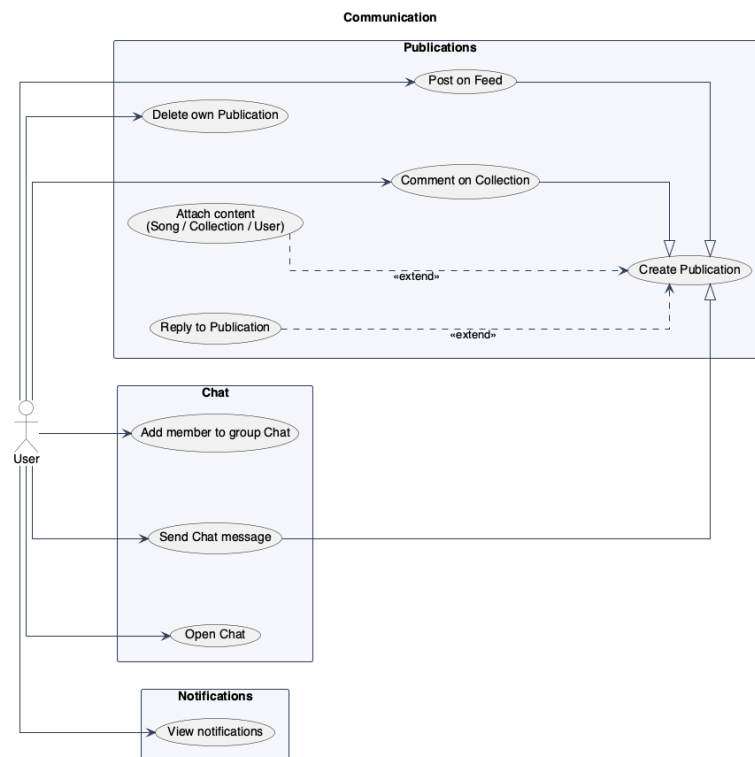


Figure C.5: Use cases: Search

**Figure C.6:** Use cases: Communication

C.2 Frontend Component Hierarchies

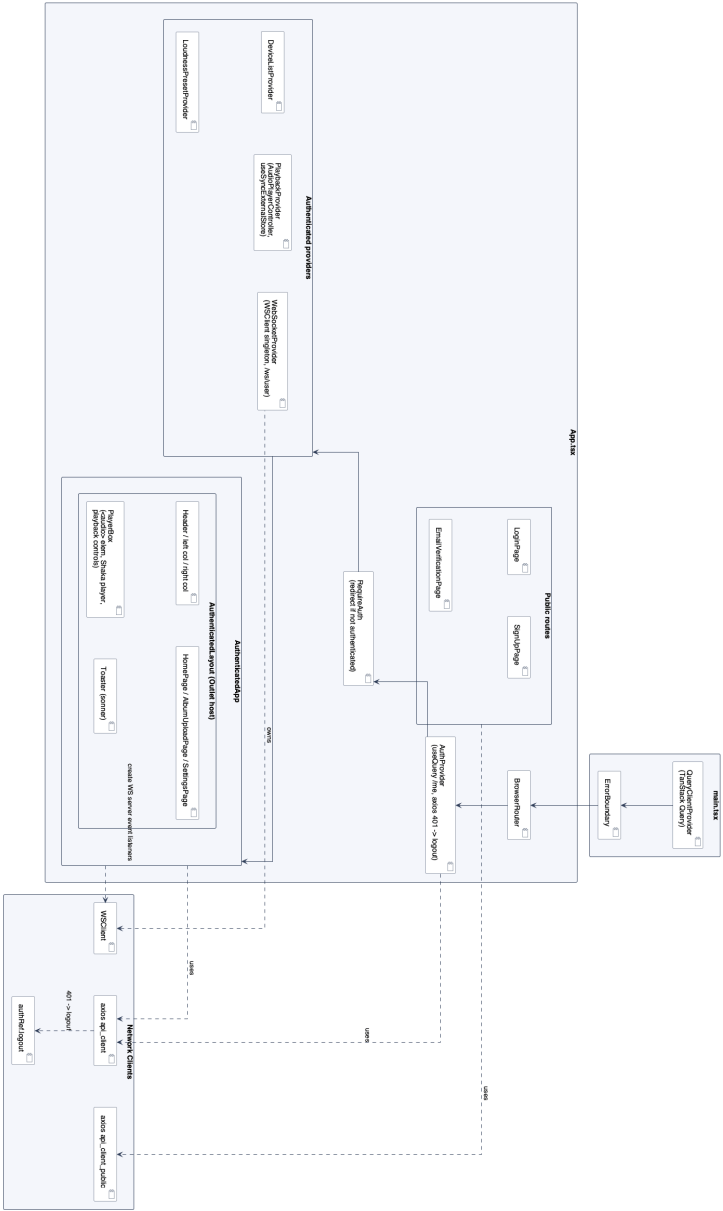


Figure C.7: Frontend component hierarchy: top level

C.3 Behavior Diagrams

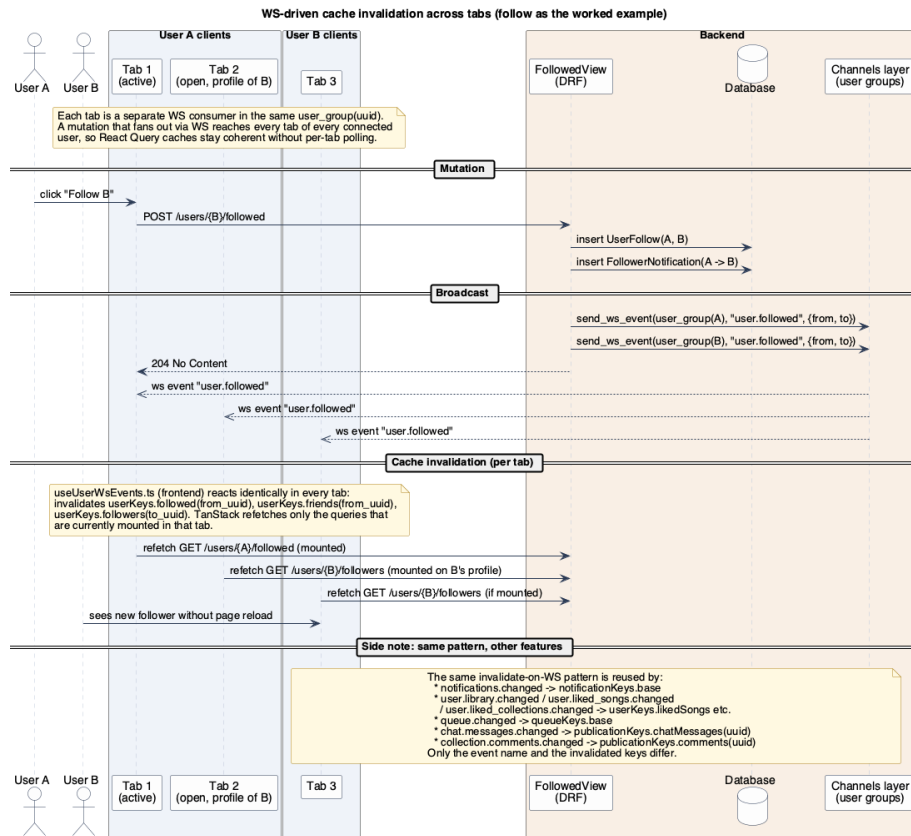


Figure C.8: WebSocket cache invalidation on follow mutation

C.4 Survey Charts

How old are you?

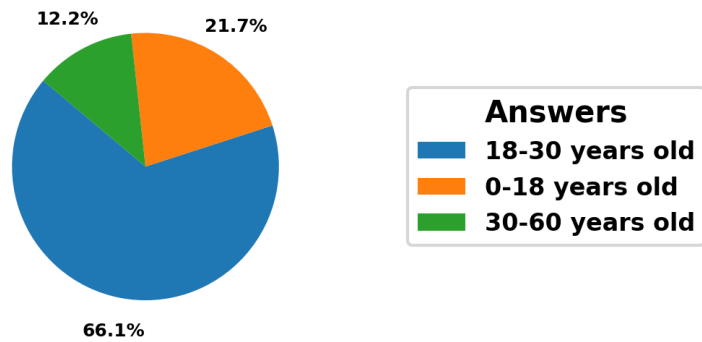


Figure C.9: Age Distribution of Respondents

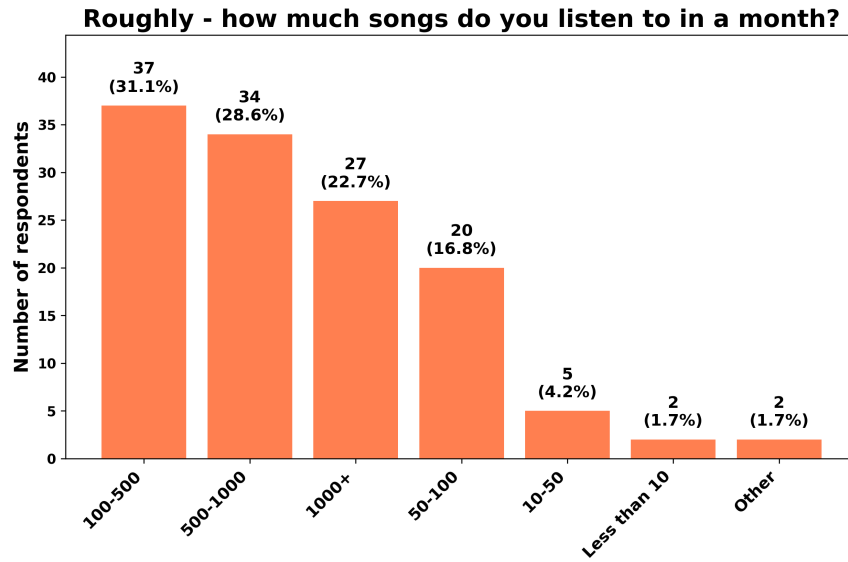


Figure C.10: Monthly Song Count per Respondent

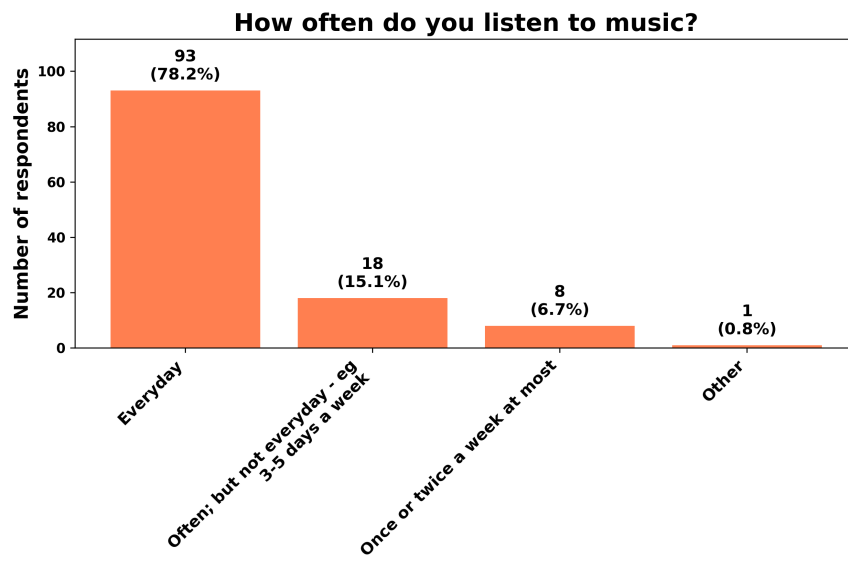


Figure C.11: Listening Frequency of Respondents

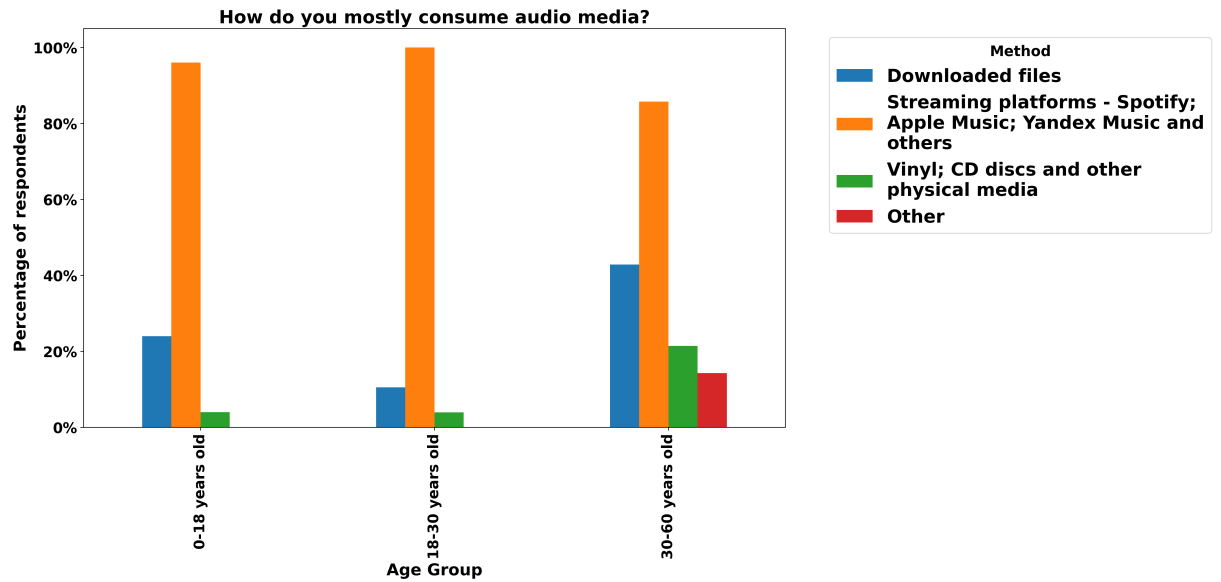


Figure C.12: Audio Media Consumption by Age Group

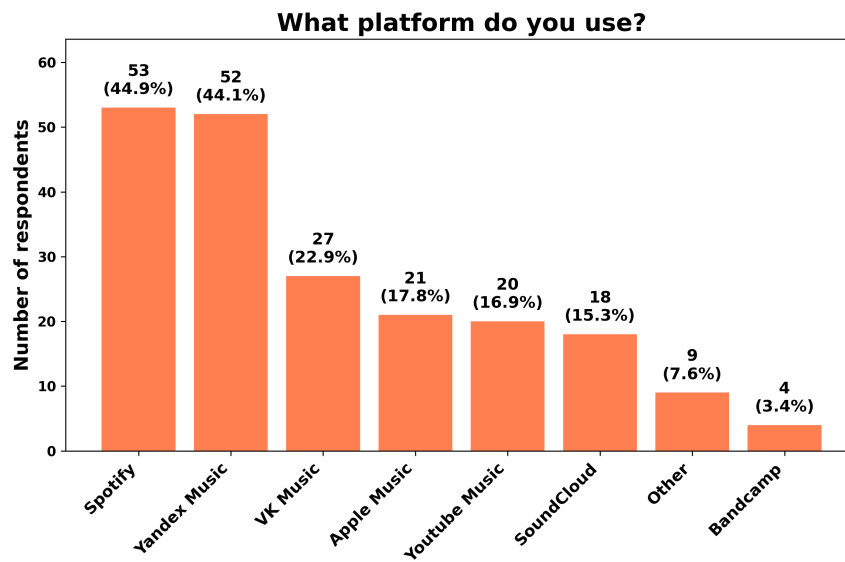


Figure C.13: Streaming Platforms Popularity

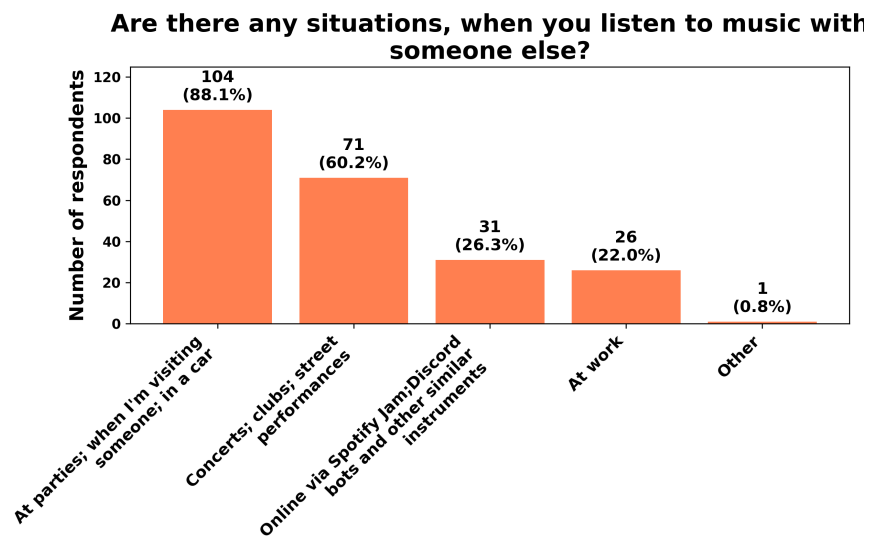


Figure C.14: Social Interaction Methods

How often do you listen to music with someone else?



Figure C.15: Social Interactions Frequency

How often do you/your friends share/discuss music?

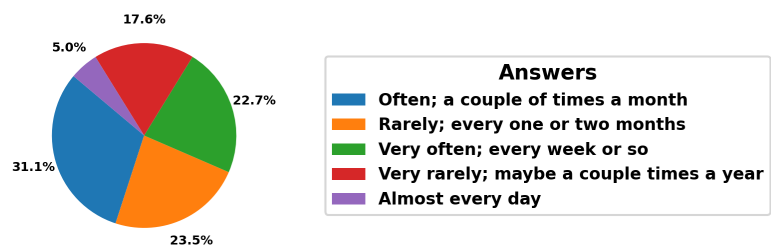


Figure C.16: Music Sharing Frequency

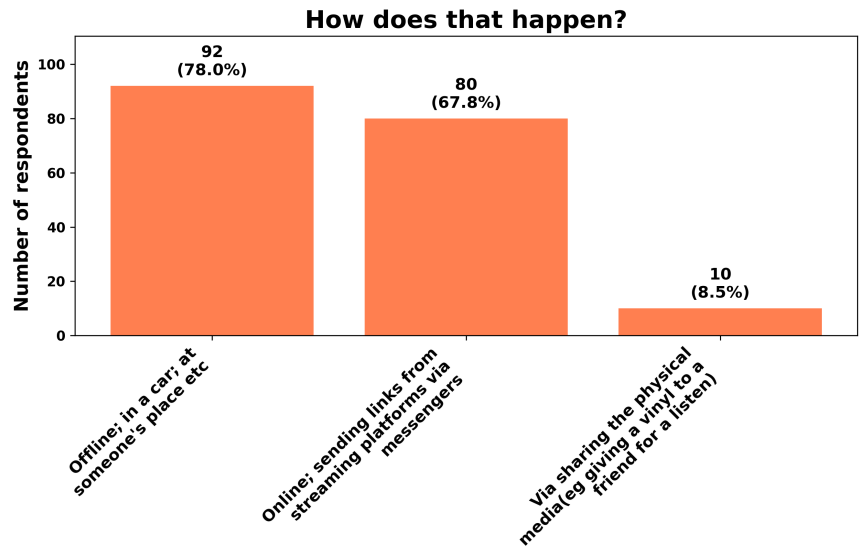


Figure C.17: Music Sharing Methods

Bibliography

- [1] *Music Streaming Market Decade Insights*. URL: <https://www.businessofapps.com/data/music-streaming-market/> (visited on 11/17/2025).
- [2] Peter J. Rentfrow. *The Role of Music in Everyday Life: Current Directions in the Social Psychology of Music*. 2012. URL: <https://compass.onlinelibrary.wiley.com/doi/abs/10.1111/j.1751-9004.2012.00434.x> (visited on 03/15/2025).
- [3] *Sharing Music: Social and Communal Aspects of Concert-Going*. URL: <https://ojs.meccsa.org.uk/index.php/netknow/article/view/428/250> (visited on 11/17/2025).
- [4] *Google Forms*. URL: <https://docs.google.com/forms/u/0/> (visited on 04/05/2025).
- [5] *Spotify Popularity*. URL: <https://simplebeen.com/music-streaming-statistics> (visited on 04/15/2025).
- [6] *Audio Usage Statistics 2024*. URL: https://www.ifpi.org/wp-content/uploads/2023/12/IFPI-Engaging-With-Music-2023_full-report.pdf (visited on 03/15/2025).
- [7] *Spotify Jam*. URL: <https://support.spotify.com/cz/article/jam/> (visited on 03/16/2025).
- [8] *Spotify Friend Activity*. URL: <https://support.spotify.com/cz/article/friend-activity/> (visited on 03/16/2025).
- [9] *Spotify Party Announcement*. URL: <https://newsroom.spotify.com/2024-09-12/fans-artists-connections-features/> (visited on 03/16/2025).
- [10] *Spotify Party*. URL: <https://support.spotify.com/us/article/listening-party/> (visited on 03/16/2025).
- [11] *Spotify Chat*. URL: <https://newsroom.spotify.com/2025-08-26/introducing-messages-a-new-way-to-share-what-you-love-on-spotify-with-friends-and-family/> (visited on 11/17/2025).
- [12] *VK Music*. URL: <https://music.vk.com/> (visited on 11/17/2025).
- [13] *SoundCloud Comments*. URL: <https://help.soundcloud.com/hc/en-us/articles/115003566008-Commenting-basics> (visited on 03/16/2025).

-
- [14] *SoundCloud Reactions*. URL: <https://help.soundcloud.com/hc/en-us/articles/31039497560731-Reactions> (visited on 03/16/2025).
 - [15] *Soundcloud Reposts*. URL: <https://help.soundcloud.com/hc/en-us/articles/115003567488-Reposting-tracks-or-playlists> (visited on 03/16/2025).
 - [16] *Rate Your Music*. URL: <https://rateyourmusic.com/discover> (visited on 03/25/2025).
 - [17] *Instagram Audio*. URL: https://help.instagram.com/664508917489819?helpref=faq_content (visited on 03/25/2025).
 - [18] Dai Clegg and Richard Barker. *Case Method Fast-Track: A Rad Approach*. URL: <https://dl.acm.org/doi/10.5555/561543> (visited on 05/05/2025).
 - [19] *Multipurpose Internet Mail Extensions Part Two: Media Types. 'octet-stream'*. URL: <https://www.rfc-editor.org/rfc/rfc2046#section-4.5.1> (visited on 11/30/2025).
 - [20] *Nginx pseudo-streaming implementation*. URL: https://nginx.org/en/docs/http/nginx_http_mp4_module.html (visited on 11/30/2025).
 - [21] *HTTP Range Requests*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Range_requests (visited on 12/07/2025).
 - [22] Issing Jochen Färber Nikolaus Döhla Stefan. *Adaptive progressive download based on the MPEG-4 file format*. URL: <https://link.springer.com/article/10.1631/jzus.2006.AS0106> (visited on 11/30/2025).
 - [23] *The MPEG-DASH Standard for Multimedia Streaming Over the Internet*. URL: <https://ieeexplore.ieee.org/abstract/document/6077864> (visited on 11/28/2025).
 - [24] *AWS CDN Caching*. URL: <https://aws.amazon.com/caching/> (visited on 12/14/2025).
 - [25] Beng Chin Ooi Quang Hieu Vu Mihai Lupu. *Peer-to-Peer Computing: Principles and Applications*. URL: https://books.google.tt/books?id=kd8_AAAAQBAJ&printsec=frontcover&source=gbv_vpt_read#v=onepage&q&f=false (visited on 01/21/2026).

-
- [26] *Basic Authentication*. URL: <https://datatracker.ietf.org/doc/html/rfc7617> (visited on 03/19/2025).
 - [27] *Overview of TLS termination and end to end TLS with Application Gateway*. URL: <https://learn.microsoft.com/en-us/azure/application-gateway/ssl-overview> (visited on 01/21/2026).
 - [28] *Session Authentication*. URL: <https://docs.djangoproject.com/en/5.2/topics/http/sessions/> (visited on 03/19/2025).
 - [29] *OAuth2*. URL: <https://datatracker.ietf.org/doc/html/rfc6749> (visited on 03/21/2025).
 - [30] *Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP*. URL: <https://datatracker.ietf.org/doc/html/rfc6202> (visited on 11/30/2025).
 - [31] *Comparison of Server to Client Communication Methods*. URL: <https://www.diva-portal.org/smash/get/diva2:1133465/FULLTEXT01.pdf?ref=hackernoon.com> (visited on 03/29/2025).
 - [32] *Server Sent Events*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events (visited on 05/02/2025).
 - [33] *Django Channels*. URL: <https://channels.readthedocs.io/en/latest/> (visited on 03/19/2025).
 - [34] *WebSockets*. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API (visited on 05/02/2025).
 - [35] *Http Caching*. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching> (visited on 05/02/2025).
 - [36] *Push API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Push_API (visited on 05/02/2025).
 - [37] *Generic Event Delivery Using HTTP Push*. URL: <https://datatracker.ietf.org/doc/html/rfc8030> (visited on 11/30/2025).
 - [38] *Service Worker API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API (visited on 11/30/2025).
 - [39] *FastAPI*. URL: <https://fastapi.tiangolo.com/> (visited on 03/18/2025).

-
- [40] *Pydantic*. URL: <https://docs.pydantic.dev/latest/> (visited on 05/01/2025).
 - [41] *FastAPI Features*. URL: <https://fastapi.tiangolo.com/features> (visited on 03/20/2025).
 - [42] *FastAPI Websockets*. URL: <https://fastapi.tiangolo.com/advanced/websockets/> (visited on 12/01/2025).
 - [43] *Django*. URL: <https://www.djangoproject.com/> (visited on 03/18/2025).
 - [44] *DRF*. URL: <https://www.django-rest-framework.org/> (visited on 03/19/2025).
 - [45] *Django Overview*. URL: <https://www.djangoproject.com/start/overview/> (visited on 12/07/2025).
 - [46] *Django Async Support*. URL: <https://docs.djangoproject.com/en/5.2/topics/async/> (visited on 01/30/2025).
 - [47] *Flask*. URL: <https://flask.palletsprojects.com/en/stable/> (visited on 03/19/2025).
 - [48] *SocketIO*. URL: <https://socket.io/> (visited on 12/01/2025).
 - [49] *Web Service Efficiency at Instagram with Python*. URL: <https://instagram-engineering.com/web-service-efficiency-at-instagram-with-python-4976d078e366> (visited on 12/07/2025).
 - [50] *Oracle Price List*. URL: <https://www.oracle.com/a/ocom/docs/corporate/pricing/technology-price-list-070617.pdf> (visited on 11/30/2025).
 - [51] *PostgreSQL Full Text Search*. URL: <https://www.postgresql.org/docs/current/textsearch.html> (visited on 12/07/2025).
 - [52] *Elasticsearch*. URL: <https://www.elastic.co/elasticsearch> (visited on 01/21/2026).
 - [53] *Sphinx Search Engine*. URL: sphinxsearch.com (visited on 01/21/2026).
 - [54] *PostgreSQL*. URL: <https://www.postgresql.org/about/> (visited on 03/26/2025).
 - [55] *Django PostgreSQL Support*. URL: <https://docs.djangoproject.com/en/5.2/ref/contrib/postgres/> (visited on 03/26/2025).
 - [56] *Postgres UUID Type*. URL: <https://www.postgresql.org/docs/current/datatype-uuid.html> (visited on 12/01/2025).

- [57] *UUIDv7*. URL: <https://www.thenile.dev/blog/uuidv7> (visited on 01/21/2026).
- [58] *Django Templates*. URL: <https://docs.djangoproject.com/en/5.2/ref/templates/language/> (visited on 01/30/2025).
- [59] *Single Page Application*. URL: <https://developer.mozilla.org/en-US/docs/Glossary/SPA> (visited on 12/07/2025).
- [60] *Clay Library*. URL: <https://github.com/nicbarker/clay> (visited on 12/01/2025).
- [61] *Yew Framework*. URL: <https://github.com/yewstack/yew> (visited on 12/01/2025).
- [62] *Framework Usage*. URL: <https://2024.stateofjs.com/en-US/libraries/> (visited on 05/01/2025).
- [63] *React*. URL: <https://react.dev/> (visited on 03/30/2025).
- [64] *StackOverflow React*. URL: <https://stackoverflow.com/questions/tagged/reactjs?tab=Newest> (visited on 12/07/2025).
- [65] *MVC*. URL: <https://developer.mozilla.org/en-US/docs/Glossary/MVC> (visited on 03/18/2025).
- [66] *Daphne GitHub Page*. URL: <https://github.com/django/daphne> (visited on 01/24/2026).
- [67] *django-filter*. URL: <https://django-filter.readthedocs.io/en/stable/> (visited on 01/24/2026).
- [68] *dj-rest-auth*. URL: <https://pypi.org/project/dj-rest-auth/> (visited on 01/24/2026).
- [69] *Redis*. URL: <https://redis.io/> (visited on 12/07/2025).
- [70] *uv*. URL: <https://docs.astral.sh/uv/> (visited on 01/24/2026).
- [71] *FFMPEG*. URL: <https://ffmpeg.org/> (visited on 05/05/2025).
- [72] *Shaka Packager*. URL: <https://shaka-project.github.io/shaka-packager/html/> (visited on 12/13/2025).
- [73] *Shaka Player*. URL: <https://github.com/shaka-project/shaka-player> (visited on 05/08/2025).
- [74] *MPEG-DASH*. URL: <https://www.mpeg.org/standards/MPEG-DASH/> (visited on 05/05/2025).
- [75] *HLS*. URL: <https://developer.apple.com/streaming/> (visited on 05/05/2025).

BIBLIOGRAPHY

- [76] *Packaging explanation*. URL: <https://shaka-project.github.io/shaka-packager/html/documentation.html> (visited on 12/13/2025).
- [77] *HTTP Live Streaming (HLS) authoring specification for Apple devices. Bitrate recommendations, section 2.25*. URL: <https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices#Bitrate-recommendations-for-Ambisonics-are-as-follows> (visited on 12/13/2025).
- [78] *EBU Evaluations of Multichannel Audio Codecs*. URL: <https://tech.ebu.ch/docs/tech/tech3324.pdf> (visited on 12/14/2025).
- [79] *Perceived Audio Quality for Streaming Stereo Music*. URL: <https://dl.acm.org/doi/pdf/10.1145/2647868.2655025> (visited on 12/13/2025).
- [80] *BACHELOR THESIS Perceived Audio Quality in Ogg Vorbis and AAC in Compressed Popular Music in Bit Rates between 96 kbit/s, 128 kbit/s and 192 kbit/s*. URL: <https://www.diva-portal.org/smash/get/diva2:1030609/FULLTEXT02.pdf> (visited on 12/14/2025).
- [81] *Spotify Audio Quality*. URL: <https://support.spotify.com/cz/article/audio-quality/> (visited on 12/14/2025).
- [82] *Web audio codec guide*. URL: https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Audio_codecs#aac_advanced_audio_coding (visited on 12/14/2025).
- [83] *python-magic Package*. URL: <https://pypi.org/project/python-magic/> (visited on 05/07/2025).
- [84] Kevin Fu et al. *Dos and Don'ts of Client Authentication on the Web*. 2001. URL: <https://users.umi.acs.umd.edu/~tudor/courses/ENEE757/Fall14/papers/Fu01.pdf> (visited on 05/12/2026).
- [85] *Linked List in SQL 1*. URL: <https://stackoverflow.com/questions/65205/linked-list-in-sql> (visited on 05/07/2025).
- [86] *Linked List in SQL 2*. URL: <https://dba.stackexchange.com/questions/46238/linked-list-in-sql-and-trees> (visited on 05/07/2025).
- [87] *Caddy*. URL: <https://caddyserver.com/> (visited on 05/08/2025).

-
- [88] *ffmpeg libfdkaac*. URL: <https://trac.ffmpeg.org/wiki/Encode/AAC> (visited on 05/06/2025).
 - [89] *Comparative Analysis of MPEG-DASH and HLS Protocols: Performance, Adaptation, and Future Directions in Adaptive Streaming*. URL: <https://link.springer.com/article/10.1007/s40031-025-01244-x#Sec4> (visited on 05/05/2025).
 - [90] *MSE*. URL: <https://www.w3.org/TR/media-source-2/> (visited on 05/05/2025).
 - [91] *MSE Availability*. URL: <https://caniuse.com/mediasource> (visited on 05/05/2025).
 - [92] *Operating System Market Share Worldwide*. URL: <https://gs.statcounter.com/os-market-share> (visited on 12/13/2025).
 - [93] *CMAF*. URL: <https://developer.apple.com/documentation/http-live-streaming/about-the-common-media-application-format-with-http-live-streaming-hls> (visited on 12/13/2025).
 - [94] *HTTP Live Streaming (HLS) authoring specification for Apple devices. Audio*. URL: <https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices#Audio> (visited on 12/13/2025).
 - [95] *FLAC*. URL: <https://xiph.org/flac/> (visited on 12/14/2025).
 - [96] *ISO/IEC 11172-3:1993 (MP3)*. URL: <https://www.iso.org/standard/22412.html> (visited on 12/14/2025).
 - [97] *ISO/IEC 13818-7:2006 (AAC)*. URL: <https://www.iso.org/standard/43345.html> (visited on 12/14/2025).
 - [98] *Opus Codec*. URL: <https://opus-codec.org/> (visited on 05/06/2025).
 - [99] *Personal Listening Test of AAC-LC and xHE-AAC at 96kbps and 128kbps*. URL: <https://hydrogenaudio.org/index.php?topic=119861.0> (visited on 01/24/2026).
 - [100] *Personal Blind Listening Test of AAC at 128 kbps (six encoders & 105 samples)*. URL: <https://hydrogenaudio.org/index.php?topic=120062.0> (visited on 01/24/2026).
 - [101] *Qaac VS FDK vs FGh vs FAAC ? Which one is better now in 2021?* URL: <https://hydrogenaudio.org/index.php/topic,120759.0.html> (visited on 01/24/2026).

BIBLIOGRAPHY

- [102] *SOFTWARE LICENSE AGREEMENT FOR macOS Tahoe 26 (section "J. Other Use Restrictions")*. URL: <https://www.apple.com/legal/sla/docs/macOSTahoe.pdf> (visited on 01/24/2026).
- [103] *Via Licensing*. URL: <https://via-la.com/licensing-programs/aac/#faqs> (visited on 12/14/2025).
- [104] *Fraunhofer FDK AAC Codec Library*. URL: <https://spdx.org/licenses/FDK-AAC.html> (visited on 01/24/2026).
- [105] *CBR*. URL: <https://learn.microsoft.com/en-us/windows/win32/wmformat/constant-bit-rate--cbr--encoding> (visited on 05/06/2025).
- [106] *VBR*. URL: https://wiki.hydrogenaudio.org/index.php?title=Variable_Bitrate (visited on 05/06/2025).
- [107] *CBR vs VBR*. URL: <https://ieeexplore.ieee.org/abstract/document/6774590> (visited on 05/06/2025).
- [108] *Movie atom ('moov')*. URL: https://developer.apple.com/documentation/quicktime-file-format/movie_atom (visited on 12/14/2025).